

Online Recruitment Fraud (ORF) Detection Using Deep Learning

*The project report submitted in partial fulfilment of the requirements
for the award of degree in*

**Bachelor of Technology
In
(Computer Science And Engineering)**

Submitted by

322506402192	KATIPAMU NEELIMA
322506402182	KARANAM DEVIKA
322506402183	K S S JAHNAVI
322506402193	KATTA SHANMUKI

Under the guidance of

Dr. A. MARY SOWJANYA

Associate Professor



Department of Computer Science and Systems Engineering

Andhra University College of Engineering (A)

Andhra University, Visakhapatnam-530003

APRIL-2026

Department Of Computer Science And Systems Engineering

Andhra University College of Engineering (A)

Andhra University, Visakhapatnam



CERTIFICATE

This is to certify that the project entitled "**Online Recruitment Fraud (ORF) Detection**" is a Bonafide work carried out by **KATIPAMU NEELIMA (322506402192)**, **KARANAM DEVIKA (322560402182)**, **K.S.S.JAHVANI (322560402183)**, **K. SHANMUKI (322560402193)** submitted in partial fulfilment of the requirements for the award of degree in Bachelor of Technology (Computer Science Engineering) in the Department of Computer Science & Systems Engineering, **Andhra University College of Engineering (A)**, **Andhra University, Visakhapatnam**, during **December 2025- April 2026**.

Project Guide

Dr. A. MARY SOWJANYA

Dept of CS & SE

AUCE, AU.

Head of the Department

Prof. V. VALLI KUMARI

Dept of CS & SE

AUCE, AU.

DECLARATION

We hereby declare that the project report entitled “**Online Recruitment Fraud (ORF) Detection**” is an original work done at Department of Computer Science and Systems Engineering, Andhra University College of Engineering (A), Andhra University, Visakhapatnam, submitted in partial fulfilment of the requirements for the award of degree in Bachelor of Technology (Computer Science and Engineering) during December 2025 – April 2026. **The same work either in part or whole is not submitted to any institution.**

Regd No.	Name of the Student	Signature
322506402192	Katipamu Neelima	
322506402182	Karanam Devika	
322506402183	K S S Jahnavi	
322506402193	Katta Shanmuki	

Place: Visakhapatnam

Date:

ACKNOWLEDGEMENT

We have immense pleasure in expressing our earnest gratitude to our esteemed Project Guide **Dr. A. Mary Sowjanya**, Associate Professor, Andhra University for her inspiring and scholarly guidance. Despite her pre-occupation with several assignments, she has been kind enough to spare her valuable time and gave us the necessary guidance at every stage of planning and constitution of this work. We express sincere gratitude for having accorded us permission to take up this project work and for helping us graciously throughout the execution of this work.

We express sincere thanks to **Prof. V. Valli Kumari**, Head of the Department, Computer Science and Systems Engineering, Andhra University College of Engineering for his keen interest and providing necessary facilities for this project study.

We would like to thank **Prof. Kuda Nageshwara Rao**, Research and Chairperson of BOS, Department of Computer Science & Systems Engineering for the valuable guidance and suggestions, keen interest and thorough encouragement extend throughout the period of project work.

We extend our sincere thanks to our academic teaching staff and nonteaching staff for their help throughout our study.

TABLE OF CONTENTS

S. No	Title	Page.No.
1	Introduction	08 - 10
1.1	Overview	08
1.2	Problem Statement	09
1.3	Objective	10
2	Literature Survey	11 – 13
3	Requirements Analysis	14 – 18
3.1	Functional requirements	14
3.2	Non-functional Requirements	14
3.3	Hardware Requirements	15
3.4	Software Requirements	16
4	System Design and Methodology	19-26
4.1	Existing System	20
4.2	Proposed System	21
4.3	System Architecture	22
4.4	UML Diagrams for the System	23
5	Implementing and Testing	27 - 82
5.1	Technology and Codes	28
5.2	Model training and Implementation	64
5.3	Testing	66
5.4	System Execution	74
5.5	Results	82
6	Conclusion	83
7	References	84 - 85

LIST OF FIGURES

S. No	Fig.no	Figure Description	Page.No.
1	i	Fake Job Detection System Architecture	22
2	ii	Use Case Diagram	23
3	iii	Class Diagram	24
4	iv	Sequence Diagram	25
5	v	Activity Diagram	26
6	vi	Model Performance Metrics	82

ABSTRACT

The rapid growth of online recruitment platforms has significantly transformed the job search process, enabling faster and more accessible hiring. However, this advancement has also led to a surge in fraudulent job postings that exploit job seekers through deceptive practices such as fake offers, upfront payment requests, identity theft, and misleading information. Detecting such fraudulent postings has become increasingly challenging due to the evolving sophistication of scams and the limitations of traditional detection methods.

This project proposes an intelligent and scalable system for detecting fake job postings using a hybrid approach that combines deep learning, rule-based analysis, and explainable artificial intelligence. The core of the system is a BERT (Bidirectional Encoder Representations from Transformers) based classifier, which captures contextual and semantic patterns in job descriptions to accurately distinguish between legitimate and fraudulent postings. The model is trained on structured datasets with preprocessing techniques such as text cleaning, tokenization, and class balancing using SMOTE to address data imbalance issues.

To enhance detection accuracy, the system integrates a rule-based fraud detection module that identifies explicit scam indicators such as requests for upfront fees, use of personal email domains, unrealistic salary promises, urgency tactics, and suspicious communication channels. This hybrid strategy ensures that both obvious and subtle fraud patterns are effectively captured.

A key feature of the system is its ability to analyze job postings from images using Optical Character Recognition (OCR). The OCR module preprocesses images and extracts text, which is then analyzed using the same detection pipeline, enabling fraud detection from screenshots and image-based job advertisements. Additionally, the system incorporates explainable AI using LIME to provide interpretable insights into model predictions, highlighting key factors influencing classification results.

The application is implemented with a user-friendly interface supporting real-time analysis, batch processing, and detailed advisory outputs. Extensive unit and integration testing ensure system reliability and performance. The proposed solution demonstrates high accuracy and robustness in detecting fraudulent job postings, providing a practical tool to enhance trust and security in online recruitment platforms.

1. INTRODUCTION

1.1 Overview

The digital transformation of recruitment has revolutionized the way employers and job seekers interact. Online job portals, professional networking platforms, and mobile applications have made job searching more accessible and efficient than ever before. However, this convenience has also introduced new risks, particularly in the form of fraudulent job postings designed to deceive and exploit users.

Fraudulent job postings often mimic legitimate opportunities, making them difficult to identify. These scams may include promises of high salaries with minimal effort, requests for upfront payments for training or equipment, use of personal email addresses instead of corporate domains, and urgency tactics to pressure applicants into quick decisions. In many cases, victims are asked to share sensitive personal or financial information, leading to identity theft or financial loss.

Traditional approaches to detecting such fraud rely on manual verification or simple keyword-based filtering systems. These methods are not only inefficient but also ineffective in handling large volumes of data and evolving scam patterns. As fraudsters adopt more sophisticated techniques, there is a growing need for intelligent systems capable of understanding context, detecting hidden patterns, and adapting to new types of fraud.

This project introduces a comprehensive solution for fake job detection using a hybrid approach. The system leverages a BERT-based deep learning model to analyze the contextual meaning of job postings, enabling it to detect subtle linguistic patterns that indicate fraud. In addition, a rule-based detection module identifies explicit fraud signals such as payment requests, suspicious communication methods, and unrealistic job claims.

To further enhance functionality, the system supports image-based job postings through OCR technology, allowing it to extract and analyze text from screenshots or scanned documents. This is particularly important as many fraudulent postings are shared in image formats to bypass traditional text-based detection systems.

Overall, this project aims to develop a robust, scalable, and user-friendly solution that not only detects fraudulent job postings with high accuracy but also provides meaningful insights to users, thereby improving safety and reliability in online recruitment ecosystems.

1.2 Problem Statement

The increasing use of online platforms for job searching has led to a significant rise in fraudulent job postings, creating serious risks for job seekers. These fake job advertisements are designed to deceive individuals by offering attractive opportunities while concealing malicious intentions such as financial fraud, identity theft, or illegal activities. Common characteristics of such postings include unrealistic salary offers, minimal qualification requirements, requests for upfront payments, use of personal communication channels, and vague or misleading company information.

One of the major challenges in addressing this issue is the lack of effective automated systems capable of accurately detecting fraudulent job postings in real time. Existing solutions primarily rely on manual moderation or basic keyword filtering techniques, which are not scalable and fail to detect sophisticated scams that use professional language and structured formats. Additionally, many fraudulent postings are shared as images or screenshots, making traditional text-based detection methods ineffective.

Another critical limitation is the absence of explainability in existing detection systems. Users often receive a classification result without understanding the reasoning behind it, reducing trust and usability. Furthermore, the imbalance between legitimate and fraudulent job data makes it difficult to train accurate machine learning models without specialized techniques.

Therefore, the problem addressed in this project is the development of an intelligent, automated, and explainable system that can:

- Accurately detect fraudulent job postings using advanced machine learning techniques
- Identify both explicit and implicit fraud patterns through a hybrid approach
- Process job postings from multiple formats, including text and images
- Provide clear and interpretable explanations for predictions
- Handle large-scale data efficiently and adapt to evolving fraud strategies

The proposed solution addresses these challenges by integrating a BERT-based deep learning model with rule-based fraud detection and OCR-based image analysis. This combination enables the system to detect a wide range of fraudulent patterns while maintaining high accuracy, scalability, and transparency, ultimately improving user safety in online recruitment environments.

1.3 Objective

The primary objective of this project is to develop a robust, high-accuracy, and transparent Hybrid Fake Job Detection System that utilizes state-of-the-art Natural Language Processing (NLP) and Computer Vision to safeguard job seekers from online recruitment fraud.

The specific goals of the system are categorized as follows:

1. Automated Fraud Identification

- **Contextual Classification:** To implement a deep learning model (BERT) capable of understanding the nuances, professional tone, and semantic structure of job advertisements to distinguish between genuine opportunities and sophisticated scams.
- **Rule-Based Filtering:** To provide an immediate "Expert Layer" that flags known fraudulent patterns (e.g., requests for upfront fees, wire transfers, or personal financial data) that deep learning models might occasionally overlook.

2. Multimodal Data Extraction

- **OCR Integration:** To enable the system to process non-textual inputs, specifically screenshots of job ads from social media or mobile apps, by integrating Optical Character Recognition (Tesseract/EasyOCR) to extract text for analysis.
- **Image Enhancement:** To automate the cleaning and preprocessing of images to ensure high extraction accuracy regardless of background noise or formatting.

3. Explainable and Trustworthy AI (XAI)

- **Decision Transparency:** To integrate LIME (Local Interpretable Model-agnostic Explanations) so that every prediction is accompanied by a visual breakdown of the "suspicious" words that led to the fraud flag.
- **User Empowerment:** To move beyond "black-box" predictions and provide users with actionable insights and risk assessments that explain the logic behind the system's warnings.

4. Technical Robustness and Scalability

- **Class Imbalance Mitigation:** To solve the "Needle in a Haystack" problem in fraud detection by utilizing SMOTE (Synthetic Minority Over-sampling Technique) to ensure the model learns from an equal representation of fraudulent and legitimate cases.
- **Seamless Integration:** To provide a production-ready Flask REST API and an interactive Streamlit Dashboard, ensuring the tool is easily deployable for both individual users and large-scale recruitment platforms.

2. LITERATURE SURVEY

The problem of detecting fraudulent job postings has become increasingly important due to the rapid growth of online recruitment platforms. Over the years, researchers and developers have proposed various techniques ranging from traditional machine learning methods to advanced deep learning and hybrid approaches. This section reviews the existing work and highlights their limitations, which motivated the development of the proposed system.

1. Traditional Machine Learning Approaches

Initial research in fake job detection focused on classical machine learning algorithms such as Logistic Regression, Naïve Bayes, Support Vector Machines (SVM), and Decision Trees. These methods relied heavily on manually engineered features extracted from job postings, including keyword frequency, presence of suspicious phrases, and structured attributes such as salary, company profile, and job requirements.

These approaches were simple to implement and computationally efficient. However, they had significant limitations. They lacked the ability to understand the context of the text, making them ineffective in detecting sophisticated fraud patterns. Additionally, their performance depended heavily on feature engineering, which required domain expertise and was not scalable for large and dynamic datasets.

2. Natural Language Processing (NLP)-Based Techniques

To overcome the limitations of traditional methods, researchers introduced Natural Language Processing techniques such as TF-IDF (Term Frequency–Inverse Document Frequency) and word embeddings like Word2Vec and GloVe. These techniques improved the representation of textual data by capturing semantic relationships between words.

Although NLP-based methods enhanced detection accuracy compared to basic machine learning models, they still struggled with understanding long-range dependencies and contextual meaning. For instance, phrases like “earn money quickly” could be legitimate in some contexts and fraudulent in others, which these models could not effectively differentiate.

3. Deep Learning and Transformer-Based Models

With advancements in deep learning, transformer-based models such as BERT (Bidirectional Encoder Representations from Transformers) have emerged as powerful tools for text classification tasks. BERT models are capable of understanding context by analyzing both left and right contexts of a word simultaneously.

In the domain of fraud detection, BERT has shown superior performance due to its ability to capture subtle linguistic patterns and contextual relationships in job descriptions. It can identify

complex fraud indicators that are not explicitly stated, making it highly effective for detecting sophisticated scams.

The proposed project adopts this approach by utilizing a BERT-based classifier as the core detection mechanism, enabling accurate and context-aware classification of job postings.

4. Handling Class Imbalance in Fraud Detection

A common challenge in fraud detection datasets is class imbalance, where legitimate job postings significantly outnumber fraudulent ones. This imbalance can lead to biased models that favor the majority class.

To address this issue, various techniques have been proposed, including oversampling, under sampling, and synthetic data generation. One of the most effective methods is SMOTE (Synthetic Minority Over-sampling Technique), which generates synthetic samples for the minority class.

The proposed system incorporates SMOTE to balance the dataset, ensuring that the model learns effectively from both legitimate and fraudulent examples, thereby improving detection performance.

5. Rule-Based Fraud Detection Systems

Rule-based systems have been widely used to detect explicit fraud patterns based on predefined rules. These systems identify common scam indicators such as requests for upfront payments, use of personal email addresses, unrealistic salary offers, and urgency phrases.

While rule-based systems are effective in detecting obvious fraud cases, they have limitations. They cannot adapt to new or evolving fraud strategies and may fail to detect subtle scams that do not explicitly contain predefined keywords.

To overcome this limitation, the proposed project integrates rule-based detection with a deep learning model, forming a hybrid system that combines the strengths of both approaches.

6. Explainable Artificial Intelligence (XAI)

One of the major challenges in modern AI systems is the lack of interpretability. Deep learning models often act as black boxes, making it difficult to understand how predictions are made.

To address this issue, Explainable AI techniques such as LIME (Local Interpretable Model-agnostic Explanations) and SHAP have been introduced. These methods provide insights into model predictions by identifying the most influential features.

In the proposed system, LIME is used to generate human-readable explanations by highlighting key words that influence the classification. This improves transparency, builds user trust, and helps users make informed decisions.

7. OCR-Based Detection for Image Data

Recent trends indicate that fraudsters often use images or screenshots to share job postings, thereby bypassing text-based detection systems. To address this, researchers have explored the use of Optical Character Recognition (OCR) techniques to extract text from images. OCR systems such as Tesseract, combined with image preprocessing techniques like grayscale conversion, noise removal, and thresholding, enable accurate text extraction from images.

The proposed system incorporates an OCR module, allowing it to process both text and image-based job postings. This significantly enhances the system's capability and robustness.

8. Hybrid Detection Systems

Recent research emphasizes the effectiveness of hybrid systems that combine multiple techniques such as machine learning, rule-based detection, NLP, and explainable AI.

Hybrid systems offer several advantages:

- Improved accuracy by leveraging multiple detection strategies
- Better generalization across different types of fraud
- Ability to detect both known and unknown fraud patterns

The proposed project follows this approach by integrating:

- BERT for contextual understanding
- Rule-based detection for explicit fraud signals
- OCR for image processing
- LIME for explainability

This combination results in a comprehensive and robust fraud detection system.

9. Research Gaps Identified

From the literature survey, the following gaps have been identified:

- Lack of systems capable of handling both text and image inputs
- Limited use of explainable AI in fraud detection
- Inability of traditional systems to detect sophisticated scams
- Poor scalability and lack of real-time analysis capabilities
- Over-reliance on either rule-based or machine learning methods alone

10. Contribution of the Proposed System

The proposed system addresses these gaps by:

- Developing a hybrid detection model combining deep learning and rule-based techniques
- Supporting multi-format input (text and images)
- Providing explainable predictions using LIME

3. REQUIREMENTS ANALYSIS

The requirements analysis of the Fake Job Detection System focuses on identifying and defining the essential functionalities, performance expectations, and resource needs required to design and implement an efficient and reliable solution. This system is developed to automatically detect fraudulent job postings using a combination of deep learning, rule-based analysis, OCR, and explainable AI techniques. The analysis is divided into functional requirements, non-functional requirements, hardware requirements, and software requirements, each addressing a specific aspect of the system.

3.1 Functional Requirements

The functional requirements describe the core operations that the system must perform. The system should allow users to input job postings in multiple formats, including plain text, batch input, and images. It must process these inputs and classify them as either legitimate or fraudulent using a hybrid detection mechanism that combines a BERT-based deep learning model and rule-based fraud detection. Additionally, the system should provide a confidence score along with each prediction. A key functional aspect is the explainability feature, where the system uses LIME to highlight important words influencing the prediction and generate a human-readable explanation. The system should also support OCR functionality to extract text from images and analyze it using the same pipeline. Furthermore, it must provide REST API endpoints for prediction, explanation, and batch processing, along with a user-friendly interface for interaction. Proper error handling should be implemented to manage invalid inputs, missing data, or system failures.

3.2 Non-Functional Requirements

The non-functional requirements define how the system should perform. The system should deliver results quickly, ensuring that predictions are generated within a few seconds to provide a smooth user experience. It must achieve high accuracy by leveraging advanced machine learning techniques and a hybrid detection approach. Scalability is another important aspect, as the system should be capable of handling large volumes of job postings and multiple user requests simultaneously. The interface should be intuitive and easy to use, even for non-technical users. Reliability is essential, meaning the system should consistently produce correct results and pass all testing stages, including unit and integration testing. Security must also be considered, ensuring safe handling of user input and preventing misuse of sensitive information. Additionally, the system should be maintainable, allowing future updates such as adding new fraud detection rules.

or improving the model. Explainability is a critical requirement, as users must understand why a job posting is classified as fraudulent or legitimate.

3.3 Hardware Requirements

1. Minimum Requirements:

To get started with model development, you can run the project on **basic hardware**. However, for faster performance, especially during training, better specifications are recommended.

- **Processor (CPU):**
 - **Minimum:** Intel i3 or equivalent
 - **Recommended:** Intel i5 or i7 (faster processing, especially for model training)
- **RAM:**
 - **Minimum:** 4 GB
 - **Recommended:** 8 GB or more (for handling larger datasets during model training)
- **Storage:**
 - **Minimum:** 10 GB of free disk space
 - **Recommended:** 20 GB+ (for saving datasets, model files, and other resources)
- **Graphics Card (GPU):**
 - **Optional (but recommended for faster training):** NVIDIA GPU (e.g., GTX 1060 or higher) for **GPU acceleration** during deep learning tasks. GPUs can significantly speed up model training (especially for BERT).
- **Operating System:**
 - **Windows** (preferred for compatibility with various tools)
 - **Mac/Linux** (can be used but requires some environment setup)

2. Recommended High-Performance Requirements:

For **better training efficiency** and faster experimentation, especially when handling large datasets or running models like **BERT**, the following hardware setup is ideal.

- **Processor (CPU):**
 - Intel i7 or better (for faster data processing and handling large models)
- **RAM:**
 - 16 GB or higher (allows faster model training and handling of large datasets)
- **Storage:**
 - SSD with at least **50 GB free space** (faster data retrieval, essential for handling large datasets and trained models)
- **Graphics Card (GPU):**

- **NVIDIA GPU** (with CUDA support) like **GTX 1660, RTX 2060, or better**
- Using a GPU accelerates the deep learning model training process significantly (especially for transformers like BERT).
- **Operating System:**
 - **Windows 10/11** or **Linux** (Linux is often preferred for machine learning tasks)

3. Cloud Environment:

For scaling up or handling very large datasets, you can use **cloud-based hardware** like **Google Colab**, **AWS EC2**, or **Microsoft Azure**. These platforms provide powerful machines (with GPUs/TPUs) for model training on a rental basis.

- **Google Colab:** Free access to GPUs for small-to-medium projects
- **AWS EC2/GCP:** Rent powerful machines with GPUs for large-scale model training

4. Additional Requirements for Image Processing (OCR):

Since the project also involves **OCR** (for image-based job postings), having additional resources for **image processing** is beneficial, such as:

- **Graphics Card (GPU):**
 - For faster image processing when using deep learning models for OCR extraction.

3.4 Software Requirements

To build and run the Fake Job Detection System, you need to have the following software tools and libraries installed on your machine. These are required for model development, data processing, OCR, and API/backend functionalities.

1. Programming Language:

- **Python** (Version 3.6 or higher)
 - Python is used because of its **rich libraries** and **wide support** for machine learning, NLP, and web development.

2. Python Libraries:

Below is a list of **essential libraries** that you will need to run the project.

➤ *Core Libraries*

- **PyTorch:** For building and running deep learning models, including the **BERT model**.
 - Install using: *pip install torch*
- **Transformers:** A library by Hugging Face to easily load and fine-tune **pretrained BERT models**.
 - Install using: *pip install transformers*

- **Scikit-learn:** For classical machine learning algorithms and tools (e.g., **SMOTE** for balancing datasets).
 - Install using: *pip install scikit-learn*
- **NumPy** and **Pandas:** For data manipulation and handling. Pandas is especially useful for working with **tabular data** (like CSVs) and **text preprocessing**.
 - Install using: *pip install numpy pandas*

➤ **OCR Libraries (For Image-Based Job Postings)**

- **EasyOCR:** For **Optical Character Recognition (OCR)**, to extract text from images.
 - Install using: *pip install easyocr*
- **Tesseract OCR:** A powerful OCR tool that works with EasyOCR for text extraction.
 - Install using:
 - **Windows:** Download tesseract and follow the instructions.
 - **Linux:** `sudo apt install tesseract-ocr`
 - **Mac:** `brew install tesseract`
- **OpenCV:** For image processing (e.g., resizing, converting images to grayscale).
 - Install using: *pip install OpenCV-python*

➤ **Deep Learning & NLP Libraries**

- **LIME:** For **explainable AI**, to explain BERT model predictions.
 - Install using: *pip install lime*
- **Flask:** For creating the **backend API**, where the system processes the requests from the frontend and returns predictions.
 - Install using: *pip install flask*
- **Streamlit:** For the **frontend interface**, allowing users to interact with the system via web interface.
 - Install using: *pip install streamlit*

➤ **Data Preprocessing and Utilities**

- **NLTK:** -For text processing tasks like tokenization, stop-word removal, etc.
 - Install using: *pip install nltk*
- **Imbalanced-learn:** -For **SMOTE** (Synthetic Minority Over-sampling Technique), used to handle class imbalance in fraud detection datasets.
 - Install using: *pip install imbalanced-learn*

3. Software Tools for Development

Text Editors/IDEs

- **VS Code** or **PyCharm**: Code editors for Python development.
- **Git**: For version control (to track changes and collaborate with others).

4. Cloud/Containerization (Optional)

For scaling and faster training, you may use cloud services or containerization tools:

- **Google Colab**: Free access to GPUs for training (cloud-based).
- **AWS / GCP / Azure**: Cloud-based servers for training with **GPU acceleration** (useful for large-scale data).
- **Docker**: For creating containers to deploy your model consistently across different environments.

5. Operating System (OS)

- **Windows** (for compatibility with various tools)
- **Linux** or **Mac** (preferred for machine learning tasks)

In summary, the requirements analysis ensures that the system is well-defined in terms of functionality, performance, and resource needs. By clearly outlining these requirements, the establishes a strong foundation for building a robust, scalable, and efficient fake job detection system capable of addressing real-world challenges in online recruitment fraud.

4. SYSTEM DESIGN AND METHODOLOGY

The system design and methodology define how the Fake Job Detection System is structured, how it processes data, and how different components interact to produce accurate and explainable results. This section includes the analysis of the existing system, the proposed system, and the overall system architecture.

System Design

The system is designed as a step-by-step process where the user gives input, the system processes it, and then shows the result.

First, the user enters a job posting either as text or uploads an image. This input is taken through a simple interface (frontend). The frontend sends this input to the backend for processing.

In the backend, the system first checks the type of input. If the input is an image, it uses OCR to convert the image into text. If the input is already text, it directly goes to the next step.

Next, the system cleans the text by removing unwanted data like symbols, links, and HTML tags. Then the cleaned text is given to the main model.

The system uses a BERT model to understand the meaning of the job posting and predict whether it is real or fake. At the same time, a rule-based system checks for common fraud keywords like “earn money fast” or “pay fee”.

Both results are combined to make a final decision. After that, the system uses LIME to explain why the job is marked as fraud or legitimate.

Types Of System Design

The project incorporates multiple types of system design approaches, making it a robust and advanced solution.

1. Architectural Design

Architectural design defines the overall structure of the system and how different components interact with each other. In this project, the system follows a layered architecture consisting of frontend, backend, processing modules, and output layers. This design ensures clear separation of responsibilities and improves system organization.

2. Modular Design

The system is divided into independent modules such as input handling, preprocessing, OCR, detection engine, and explanation module. Each module performs a specific function and can be developed, tested, and maintained independently. This modular approach improves flexibility and simplifies debugging.

3. Client-Server Design

The system follows a client-server architecture where the frontend (Streamlit) acts as the client and the backend (Flask) acts as the server. The client sends requests to the server, and the server processes them and returns results. This separation improves scalability and allows the system to handle multiple users.

4. HYBRID SYSTEM Design

The project uses a hybrid approach by combining different techniques, including deep learning (BERT) and rule-based logic. This design enhances accuracy by capturing both complex patterns (using BERT) and explicit fraud indicators (using rules).

5. Data Flow Design

Data flow design represents how data moves through the system. In this project, the flow starts from user input, passes through preprocessing, detection, and explanation stages, and ends with output generation. This structured flow ensures smooth data processing.

6. Pipeline Design

The system follows a pipeline-based design where data is processed step-by-step in a sequential manner. Each stage, such as OCR, preprocessing, prediction, and explanation, depends on the output of the previous stage. This ensures organized and efficient processing.

7. Machine Learning System Design

The project includes a machine learning pipeline that involves data preprocessing, model loading, prediction, and evaluation. The BERT model is used as the core component for classification, making the system intelligent and adaptive.

8. Explainable Ai Design

The system incorporates explainable AI using LIME, which makes the predictions interpretable. This design improves user trust and allows users to understand the reasoning behind the system's decisions.

4.1. Existing System

In current online recruitment platforms, fraud detection is either minimal or relies heavily on manual verification. Most platforms use basic filtering mechanisms such as keyword matching or simple rule-based checks to identify suspicious job postings. These systems are limited in their ability to detect complex fraud patterns, as they cannot understand the contextual meaning of job descriptions.

Traditional systems often depend on human moderators to review job postings, which is time-consuming, inefficient, and not scalable for large datasets. Additionally, many fraud detection mechanisms fail to adapt to evolving scam strategies, as fraudsters continuously modify their

techniques to bypass detection. For example, scammers may use professional language, structured job descriptions, or indirect fraud indicators that are difficult to identify using basic methods.

Another major limitation of existing systems is their inability to process non-textual data. Many fraudulent job postings are shared as images or screenshots, which cannot be analyzed by text-based detection systems. Furthermore, most existing solutions lack transparency, as they do not provide explanations for their predictions, making it difficult for users to trust the results.

Overall, the existing systems suffer from limitations such as low accuracy, lack of scalability, inability to detect sophisticated scams, absence of explainability, and restricted input handling.

4.2 Proposed System

To overcome the limitations of existing systems, the proposed system introduces a hybrid, intelligent, and scalable solution for detecting fraudulent job postings. The system combines deep learning, rule-based analysis, OCR, and explainable AI to provide accurate and transparent results. At the core of the system is a BERT-based deep learning model, which analyzes the contextual meaning of job postings. Unlike traditional models, BERT understands language semantics and identifies subtle fraud patterns that are not explicitly stated. This allows the system to detect sophisticated scams that mimic legitimate job postings.

In addition to the deep learning model, the system incorporates a rule-based detection module that identifies explicit fraud indicators such as requests for upfront payments, use of personal email addresses, unrealistic salary promises, and urgency phrases. This hybrid approach ensures that both obvious and hidden fraud patterns are effectively captured.

The system also includes an OCR module that enables it to process job postings from images. By extracting text from images and feeding it into the detection pipeline, the system overcomes the limitations of traditional text-only systems.

A key feature of the proposed system is its explainability, achieved using LIME. This component highlights important words and features that influence the prediction, providing users with a clear understanding of why a job posting is classified as fraudulent or legitimate.

The system is implemented as a complete application with:

- A backend API (Flask) for handling predictions and explanations
- A frontend interface (Streamlit) for user interaction
- Support for real-time analysis, batch processing, and image-based detection

Thus, the proposed system provides a comprehensive, accurate, and user-friendly solution for detecting fraudulent job postings.

4.3 System Architecture

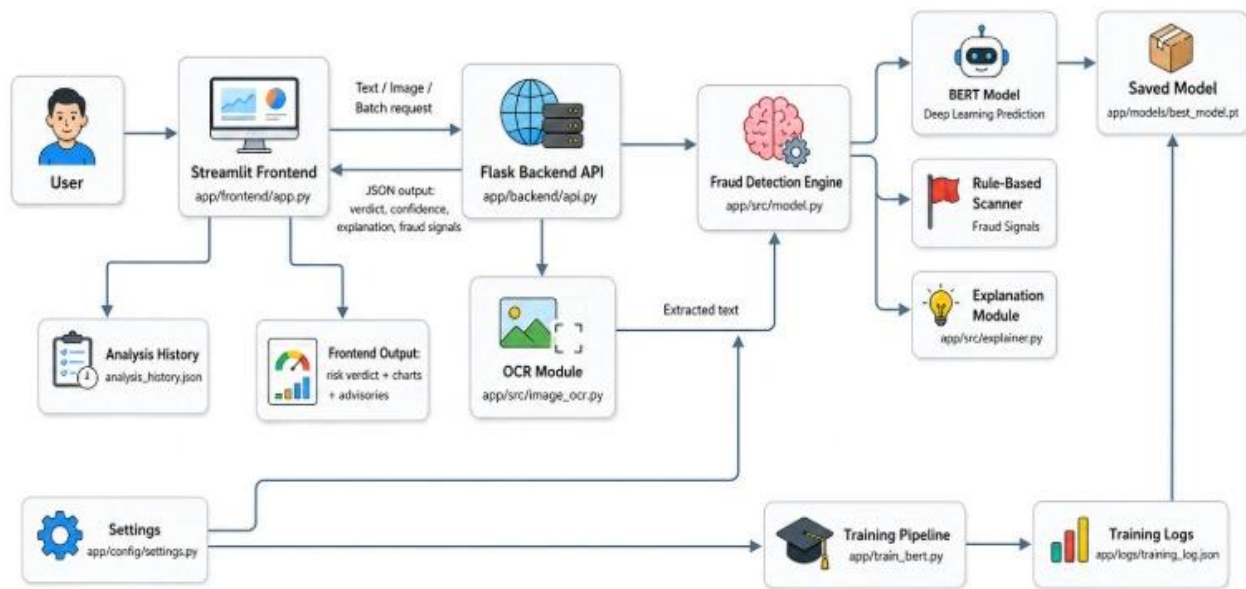


Fig i : Fake Job Detection System Architecture

The Fake Job Detection System follows a modular and hybrid architecture that integrates multiple components such as frontend, backend, preprocessing, machine learning, OCR, and explainable AI to provide accurate and reliable fraud detection. The system begins with the user, who provides input in the form of job posting text or images. This input is received through a user-friendly frontend interface built using Streamlit, which allows users to enter text, upload images, and initiate analysis.

The frontend communicates with the backend through HTTP requests. The backend, implemented using Flask, acts as the central controller that manages all operations, including prediction, explanation, and batch processing. It receives input data from the frontend and routes it to the appropriate processing modules.

The input processing module determines whether the input is text or an image. If the input is an image, it is passed to the OCR module, where text is extracted using image processing techniques. The extracted text, along with direct text inputs, is then forwarded to the preprocessing module.

The preprocessing module prepares the data for analysis by cleaning the text, removing unwanted elements such as HTML tags, URLs, and special characters, and converting it into a structured format. The text is then tokenized using a BERT tokenizer, making it suitable for the deep learning model.

The core of the system is the detection engine, which consists of two main components: a BERT-based classifier and a rule-based detection module. The BERT model analyzes the contextual

meaning of the text and generates prediction probabilities, while the rule-based module identifies explicit fraud indicators such as payment requests and urgency phrases. The outputs from both components are combined to produce the final classification.

After prediction, the system uses the explainability module based on LIME to generate human-readable explanations. This module highlights important words and features that influenced the prediction, helping users understand the reasoning behind the result.

Finally, the output module displays the prediction (fraudulent or legitimate), confidence score, and explanation to the user through the frontend interface. This architecture ensures that the system is scalable, efficient, and capable of handling multiple input formats while providing accurate and transparent results.

4.4 Uml Diagrams For The System

UML (Unified Modeling Language) diagrams help visualize the structure and behavior of the system. For this project, the most important UML diagrams are:

Use Case Diagram

Class Diagram

Sequence Diagram

Activity Diagram

1. Use Case Diagram

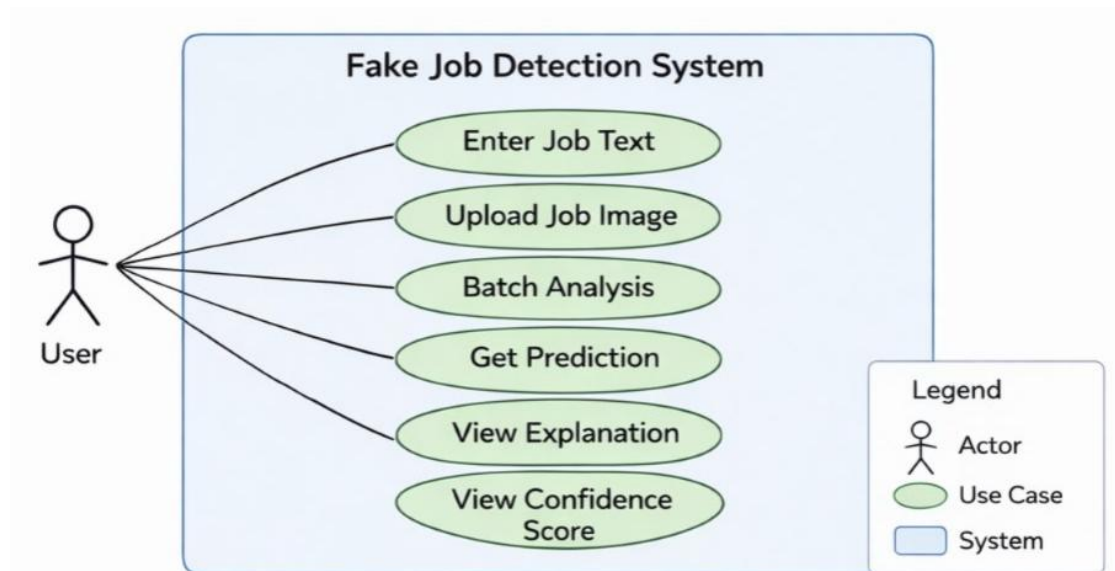


Fig ii : Use Case Diagram

The Use Case Diagram represents the interaction between the user and the Fake Job Detection System. In this system, the primary actor is the user, who interacts with the application to analyze

job postings. The user can enter job text manually, upload job images for OCR-based analysis, or perform batch analysis for multiple job postings. Once the input is provided, the system processes the data and returns a prediction indicating whether the job is legitimate or fraudulent, along with a confidence score. Additionally, the system provides an explanation of the prediction using explainable AI techniques. This diagram highlights the core functionalities of the system from the user's perspective and demonstrates how users interact with different features of the application.

2. Class Diagram

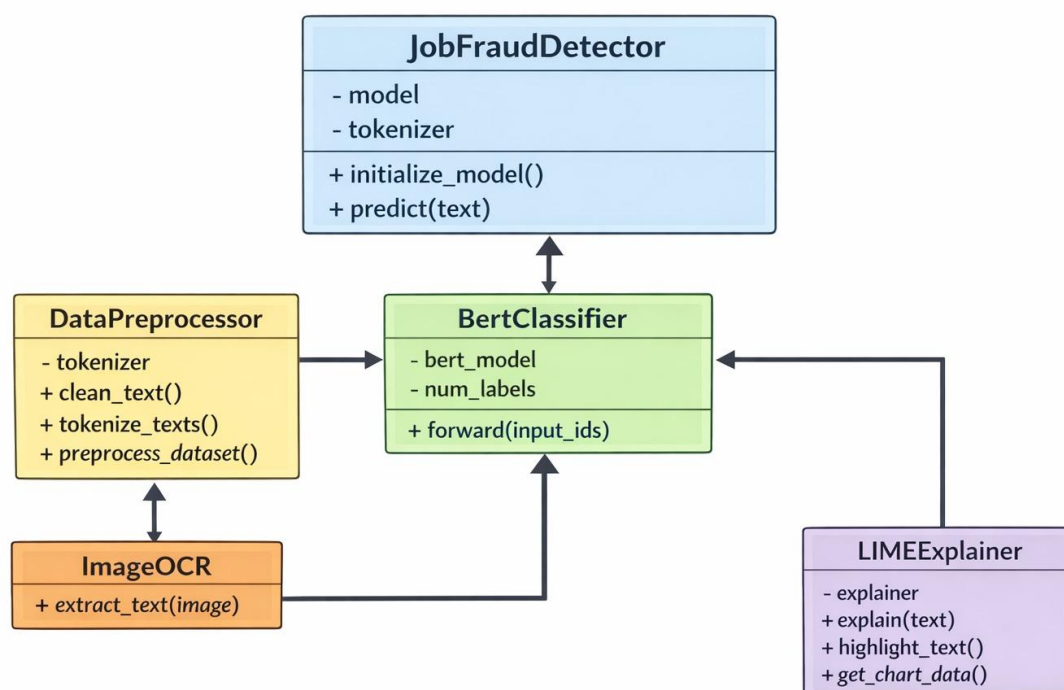


Fig iii : Class Diagram

The Class Diagram illustrates the structure of the system by showing the main classes and their relationships. The central class is the JobFraudDetector, which acts as the main controller for prediction. It interacts with the Bert Classifier to perform deep learning-based classification of job postings. The Data Preprocessor class is responsible for cleaning and preparing the text data before it is fed into the model. The ImageOCR class handles image-based inputs by extracting text from job posting images. The LIME Explainer class is used to generate explanations for predictions by identifying important features that influence the classification. Together, these classes form a modular architecture where each component performs a specific task, ensuring flexibility, maintainability, and scalability of the system.

3. Sequence Diagram

User → Frontend → Backend → Preprocessor → Model → LIME → Output

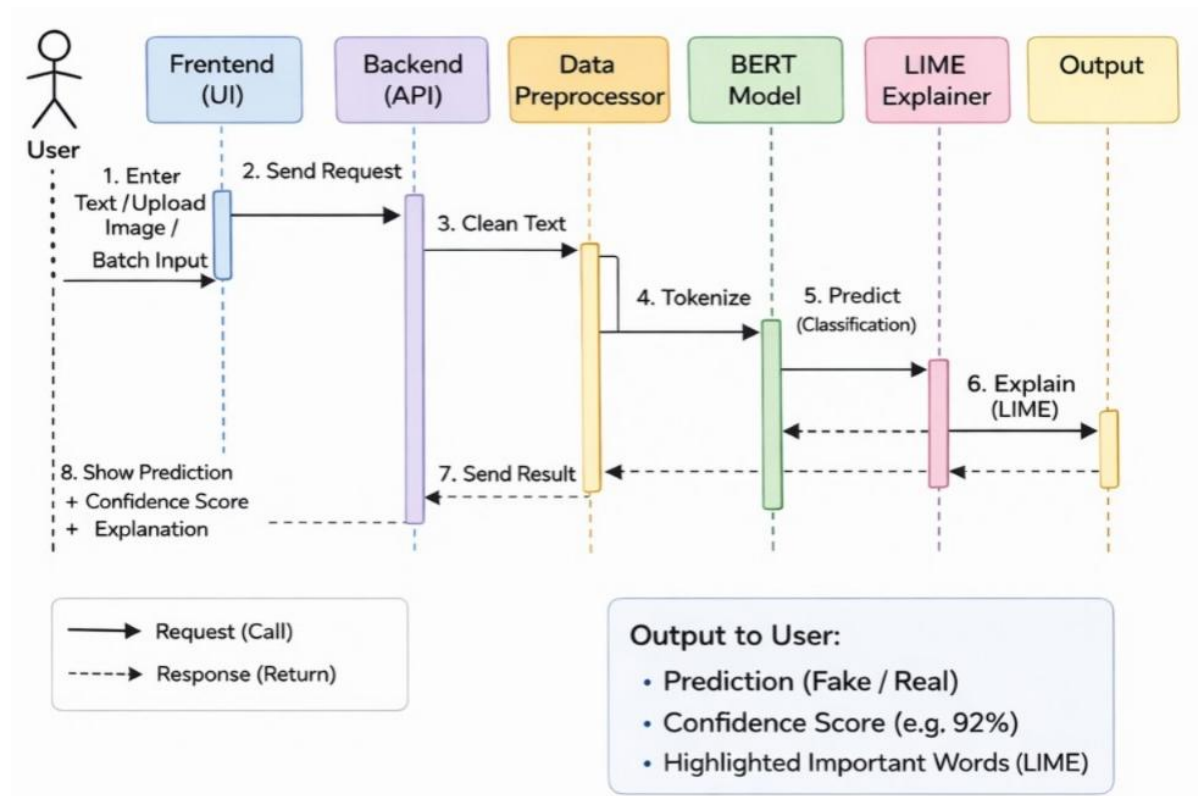


Fig iv : Sequence Diagram

The Sequence Diagram represents the step-by-step interaction between different components of the system during execution. When the user enters job posting text, the frontend sends the request to the backend API. The backend then passes the input to the preprocessing module, where the text is cleaned and tokenized. The processed data is then sent to the BERT model for prediction. After obtaining the prediction, the system invokes the LIME module to generate an explanation for the result. Finally, the backend sends the prediction, confidence score, and explanation back to the frontend, which displays the output to the user. This diagram clearly shows the flow of data and control between different modules in the system.

4. Activity Diagram

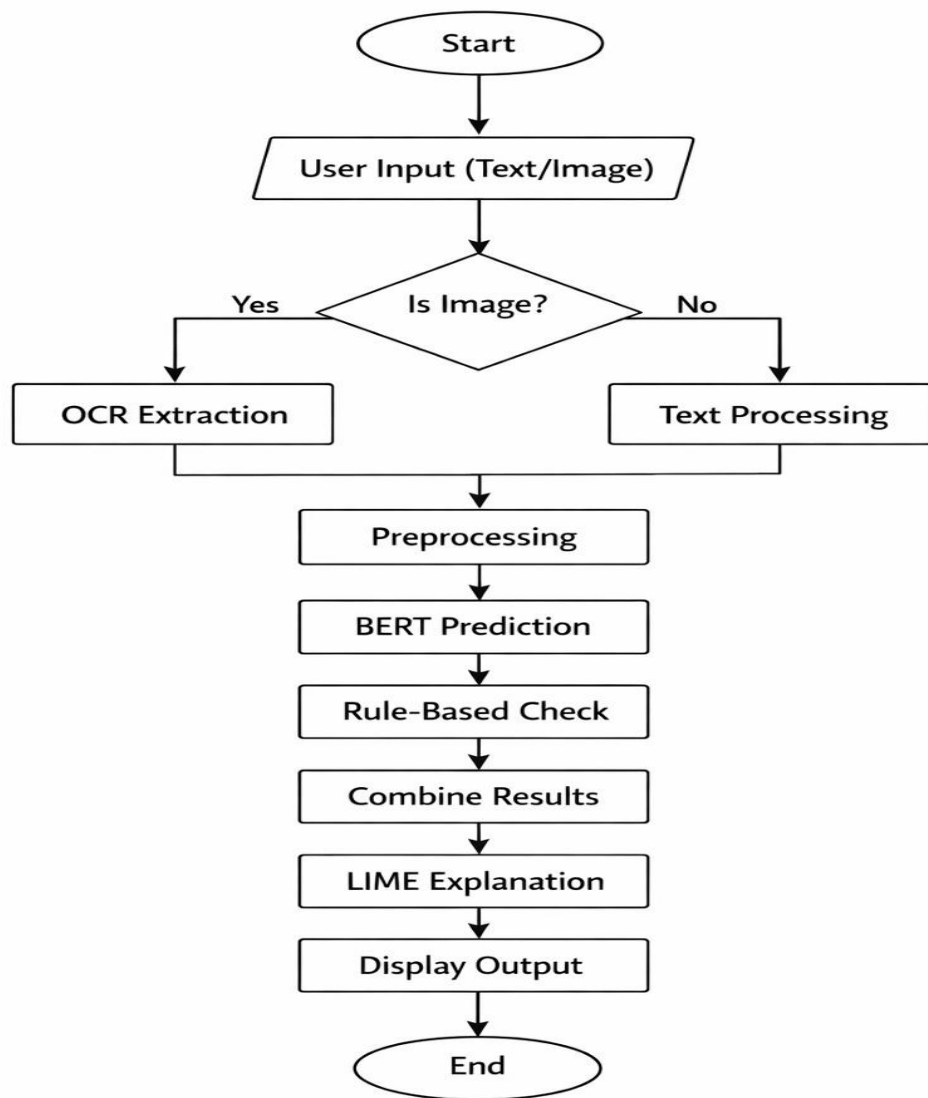


Fig v : Activity Diagram

The Activity Diagram illustrates the workflow of the system from input to output. The process begins when the user provides input in the form of text or an image. If the input is an image, it is first processed using OCR to extract text. The extracted or input text is then passed to the preprocessing module, where it is cleaned and prepared for analysis. The processed data is fed into the BERT model to generate a prediction. Simultaneously, the rule-based module checks for explicit fraud indicators. The results from both components are combined to produce the final classification. The system then generates an explanation using LIME and displays the prediction, confidence score, and explanation to the user. This diagram provides a clear understanding of the system's workflow and decision-making process.

5. IMPLEMENTATION AND TESTING

5.1 Technology and Codes

The Online Recruitment Fraud Detection System is developed using a combination of modern technologies that support machine learning, natural language processing, web development, and image processing. The core programming language used in the project is Python, which is widely preferred due to its simplicity, flexibility, and strong ecosystem of libraries for artificial intelligence and data analysis.

For implementing the deep learning model, the project uses PyTorch, a powerful open-source deep learning framework that allows efficient model training and inference. Along with PyTorch, the Transformers library by Hugging Face is used to load and utilize the pre-trained BERT (Bidirectional Encoder Representations from Transformers) model. BERT plays a crucial role in understanding the contextual meaning of job descriptions and classifying them as fraudulent or legitimate.

For data handling and preprocessing, libraries such as NumPy and Pandas are used. These libraries help in managing datasets, performing numerical operations, and transforming data into suitable formats for model training. Additionally, NLTK (Natural Language Toolkit) is used for text preprocessing tasks such as tokenization, stop-word removal, and text normalization.

To handle image-based job postings, the project incorporates OCR (Optical Character Recognition) technologies such as Tesseract OCR and EasyOCR. These tools extract textual information from images, enabling the system to analyze job postings that are shared as images. OpenCV is also used for image preprocessing tasks like noise removal and grayscale conversion, improving OCR accuracy.

The system also includes LIME (Local Interpretable Model-agnostic Explanations) to provide explainability for model predictions. LIME helps in identifying and highlighting the important words that influence the classification, making the system transparent and trustworthy.

For backend development, the project uses Flask, a lightweight web framework that handles API requests, processes input data, and communicates with the machine learning model. The frontend is built using Streamlit, which provides an interactive and user-friendly interface for entering job descriptions, uploading images, and viewing results.

Additionally, Scikit-learn is used for model evaluation and performance measurement, while Imbalanced-learn is used to handle class imbalance using techniques like SMOTE. For development and version control, tools such as Visual Studio Code (VS Code) and Git are used.

Overall, the project integrates multiple technologies including deep learning frameworks, NLP libraries, OCR tools, and web development frameworks to create a comprehensive, efficient, and explainable system for detecting fraudulent job postings.

1. Frontend

app.py

```
import streamlit as st
import requests
import re
import json
import os
import base64
from datetime import datetime
from pathlib import Path
import plotly.graph_objects as go
try:
    from PIL import Image
    import io
    IMAGE_SUPPORT = True
except ImportError:
    IMAGE_SUPPORT = False
API_URL = "http://localhost:5000"
HISTORY_FILE = Path("analysis_history.json")
st.set_page_config(
    page_title="JobGuard AI",
    page_icon="🛡️",
    layout="wide",
    initial_sidebar_state="collapsed"
)
def load_history():
    if HISTORY_FILE.exists():
        try:
            with open(HISTORY_FILE, "r") as f:
                return json.load(f)
        except Exception:
```

```

        return []
    return []
def save_history(history):
    with open(HISTORY_FILE, "w") as f:
        json.dump(history, f, indent=2)
def add_to_history(text, result, analysis_type="quick"):
    history = load_history()
    entry = {
        "id": datetime.now().strftime("%Y%m%d%H%M%S%f"),
        "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "text_preview": text[:120].strip() + ("..." if len(text) > 120 else ""),
        "full_text": text,
        "analysis_type": analysis_type,
        "label": result.get("prediction", {}).get("label", "Unknown"),
        "is_fraudulent": result.get("prediction", {}).get("is_fraudulent", False),
        "confidence": result.get("prediction", {}).get("confidence", 0),
        "fraud_signals": result.get("prediction", {}).get("fraud_signals", []),
        "probabilities": result.get("prediction", {}).get("probabilities", {}),
    }
    history.insert(0, entry)
    history = history[:100] # keep last 100
    save_history(history)
    return entry
def clear_history():
    save_history([])
def check_api():
    try:
        r = requests.get(f"{API_URL}/api/health", timeout=5)
        return r.status_code == 200
    except Exception:
        return False
def predict_job(text):
    try:
        r = requests.post(f"{API_URL}/api/predict", json={"text": text}, timeout=30)

```

```

        return r.json() if r.status_code == 200 else None
    except Exception as e:
        st.error(f"API error: {e}")
        return None
def get_explanation(text):
    try:
        r = requests.post(f"{API_URL}/api/explain", json={"text": text}, timeout=120)
        return r.json() if r.status_code == 200 else None
    except Exception as e:
        st.error(f"API error: {e}")
        return None
def explain_image_api(image_data: bytes):
    try:
        image_b64 = base64.b64encode(image_data).decode("utf-8")
        r = requests.post(f"{API_URL}/api/explain-image", json={"image": image_b64},
        timeout=180)
        if r.status_code == 200:
            return r.json()
        err = r.json() if r.content else {}
        st.error(f"Error: {err.get('error', 'Unknown error')}")
        return None
    except Exception as e:
        st.error(f"API error: {e}")
        return None
def batch_predict(texts):
    try:
        r = requests.post(f"{API_URL}/api/batch-predict", json={"texts": texts}, timeout=120)
        return r.json() if r.status_code == 200 else None
    except Exception as e:
        st.error(f"API error: {e}")
        return None
def load_css():
    st.markdown("""
    <style>

```

```

@import
url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;400;500;600;700&family=
DM+Mono:wght@400;500&family=Syne:wght@700;800&display=swap');
* { font-family: 'DM Sans', sans-serif; }
h1, h2, h3, .hero-title { font-family: 'Syne', sans-serif !important; }
code, pre { font-family: 'DM Mono', monospace !important; }
.stApp {
  background: #070B14;
  color: #E8EAF0;
}
.block-container {
  padding: 0 3rem 3rem 3rem !important;
  max-width: 1440px !important;
}
.nav-bar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 1.25rem 0;
  border-bottom: 1px solid rgba(255,255,255,0.06);
  margin-bottom: 2.5rem;
}
.nav-logo {
  display: flex;
  align-items: center;
  gap: 0.75rem;
  font-family: 'Syne', sans-serif;
  font-size: 1.4rem;
  font-weight: 800;
  color: #fff;
  letter-spacing: -0.02em;
}
.nav-logo-icon {
  width: 36px; height: 36px;

```

```

background: linear-gradient(135deg, #3B82F6, #8B5CF6);
border-radius: 10px;
display: inline-flex;
align-items: center;
justify-content: center;
font-size: 1.1rem;
}
.nav-status-dot {
display: inline-block;
width: 8px; height: 8px;
border-radius: 50%;
margin-right: 6px;
animation: pulse 2s infinite;
}
.dot-green { background: #10B981; box-shadow: 0 0 8px #10B981; }
.dot-red { background: #EF4444; box-shadow: 0 0 8px #EF4444; }
@keyframes pulse {
0%,100% { opacity: 1; } 50% { opacity: 0.5; }
}
.nav-status {
font-size: 0.82rem;
color: #94A3B8;
display: flex;
align-items: center;
}
.hero-section {
position: relative;
border-radius: 24px;
padding: 4rem 3rem;
text-align: center;
margin-bottom: 3rem;
overflow: hidden;
background: linear-gradient(135deg, #0F1B35 0%, #1a1040 50%, #0d1f3c 100%);
border: 1px solid rgba(59,130,246,0.2);

```

```

}
.hero-section::before {
  content: "";
  position: absolute;
  inset: 0;
  background: radial-gradient(ellipse 80% 60% at 50% -10%, rgba(59,130,246,0.18) 0%,
transparent 70%),
    radial-gradient(ellipse 50% 40% at 80% 110%, rgba(139,92,246,0.12) 0%, transparent
60%);
  pointer-events: none;
}
.hero-grid {
  position: absolute;
  inset: 0;
  background-image:
    linear-gradient(rgba(59,130,246,0.04) 1px, transparent 1px),
    linear-gradient(90deg, rgba(59,130,246,0.04) 1px, transparent 1px);
  background-size: 48px 48px;
  pointer-events: none;
}
.hero-badge {
  display: inline-block;
  background: rgba(59,130,246,0.15);
  border: 1px solid rgba(59,130,246,0.35);
  color: #93C5FD;
  font-size: 0.75rem;
  font-weight: 600;
  letter-spacing: 0.1em;
  text-transform: uppercase;
  padding: 0.3rem 1rem;
  border-radius: 999px;
  margin-bottom: 1.25rem;
  position: relative;
}

```

```
.hero-title {
  font-family: 'Syne', sans-serif !important;
  font-size: 3.8rem;
  font-weight: 800;
  color: #fff;
  line-height: 1.1;
  letter-spacing: -0.04em;
  margin-bottom: 1rem;
  position: relative;
}

.hero-title span {
  background: linear-gradient(90deg, #60A5FA, #A78BFA);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}

.hero-sub {
  font-size: 1.1rem;
  color: #94A3B8;
  max-width: 540px;
  margin: 0 auto 2rem;
  line-height: 1.7;
  position: relative;
}

.hero-stats {
  display: flex;
  justify-content: center;
  gap: 3rem;
  position: relative;
}

.hero-stat-num {
  font-family: 'Syne', sans-serif;
  font-size: 1.6rem;
  font-weight: 800;
  color: #fff;
```

```

}
.hero-stat-label { font-size: 0.8rem; color: #64748B; }
.section-header {
  font-family: 'Syne', sans-serif !important;
  font-size: 0.7rem;
  font-weight: 700;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color: #3B82F6;
  margin-bottom: 0.5rem;
}
.section-title {
  font-family: 'Syne', sans-serif !important;
  font-size: 1.15rem;
  font-weight: 700;
  color: #F1F5F9;
  margin-bottom: 1.25rem;
}
.glass-card {
  background: rgba(15, 23, 42, 0.7);
  border: 1px solid rgba(255,255,255,0.07);
  border-radius: 20px;
  padding: 1.75rem;
  margin-bottom: 1.25rem;
  backdrop-filter: blur(12px);
}
.stTextArea textarea {
  min-height: 300px !important;
  font-size: 0.95rem !important;
  line-height: 1.75 !important;
  padding: 1.25rem !important;
  border-radius: 14px !important;
  border: 1.5px solid rgba(59,130,246,0.2) !important;
  background: rgba(7,11,20,0.9) !important;

```

```

    color: #CBD5E1 !important;
    font-family: 'DM Sans', sans-serif !important;
    transition: border-color 0.2s ease !important;
    resize: vertical !important;
}
.stTextArea textarea:focus {
    border-color: rgba(59,130,246,0.6) !important;
    box-shadow: 0 0 0 3px rgba(59,130,246,0.12) !important;
}
.stTextArea textarea::placeholder { color: #334155 !important; }
.stTextArea label { display: none !important; }
.stButton > button {
    border-radius: 10px !important;
    font-family: 'DM Sans', sans-serif !important;
    font-weight: 600 !important;
    font-size: 0.9rem !important;
    padding: 0.65rem 1.5rem !important;
    transition: all 0.2s ease !important;
    border: none !important;
}
.stButton > button[kind="primary"],
.stButton > button:first-child {
    background: linear-gradient(135deg, #3B82F6 0%, #6D28D9 100%) !important;
    color: white !important;
    box-shadow: 0 4px 20px rgba(59,130,246,0.25) !important;
}
.stButton > button:hover {
    transform: translateY(-2px) !important;
    box-shadow: 0 8px 28px rgba(59,130,246,0.35) !important;
}
.verdict-safe {
    background: linear-gradient(135deg, rgba(16,185,129,0.12) 0%, rgba(5,150,105,0.06) 100%);
    border: 1.5px solid rgba(16,185,129,0.35);
    border-radius: 18px;

```

```

padding: 2rem;
text-align: center;
margin-bottom: 1.25rem;
}
.verdict-fraud {
background: linear-gradient(135deg, rgba(239,68,68,0.12) 0%, rgba(220,38,38,0.06) 100%);
border: 1.5px solid rgba(239,68,68,0.35);
border-radius: 18px;
padding: 2rem;
text-align: center;
margin-bottom: 1.25rem;
}
.verdict-icon { font-size: 2.8rem; margin-bottom: 0.75rem; }
.verdict-label {
font-family: 'Syne', sans-serif !important;
font-size: 1.5rem;
font-weight: 800;
color: #fff;
}
.verdict-sub { font-size: 0.85rem; color: #94A3B8; margin-top: 0.4rem; }
.verdict-score {
font-family: 'DM Mono', monospace;
font-size: 2.2rem;
font-weight: 500;
margin-top: 1rem;
color: #fff;
}
.mini-stat-box {
background: rgba(15,23,42,0.8);
border: 1px solid rgba(255,255,255,0.07);
border-radius: 14px;
padding: 1rem;
text-align: center;
}

```

```

.mini-stat-val {
  font-family: 'Syne', sans-serif;
  font-size: 1.5rem;
  font-weight: 800;
  color: #fff;
}
.mini-stat-lbl { font-size: 0.75rem; color: #64748B; margin-top: 0.2rem; }
.signal-pill {
  display: inline-block;
  background: rgba(251,191,36,0.1);
  border: 1px solid rgba(251,191,36,0.3);
  color: #FCD34D;
  font-size: 0.78rem;
  font-weight: 500;
  padding: 0.28rem 0.8rem;
  border-radius: 999px;
  margin: 0.2rem;
}
.stTabs [data-baseweb="tab-list"] {
  background: rgba(15,23,42,0.6);
  border-radius: 12px;
  padding: 4px;
  gap: 4px;
  border: 1px solid rgba(255,255,255,0.06);
}
.stTabs [data-baseweb="tab"] {
  border-radius: 8px !important;
  color: #64748B !important;
  font-size: 0.85rem !important;
  font-weight: 500 !important;
  padding: 0.5rem 1.2rem !important;
  background: transparent !important;
}
.stTabs [aria-selected="true"] {

```

```
background: rgba(59,130,246,0.2) !important;
color: #93C5FD !important;
}
.page-nav {
  display: flex;
  gap: 0.5rem;
  margin-bottom: 2.5rem;
}
.page-tab {
  padding: 0.5rem 1.5rem;
  border-radius: 10px;
  font-size: 0.875rem;
  font-weight: 600;
  cursor: pointer;
  border: 1px solid rgba(255,255,255,0.08);
  color: #64748B;
  background: rgba(15,23,42,0.5);
  text-decoration: none;
  transition: all 0.2s ease;
}
.page-tab.active {
  background: rgba(59,130,246,0.15);
  border-color: rgba(59,130,246,0.4);
  color: #93C5FD;
}
.history-card {
  background: rgba(15,23,42,0.7);
  border: 1px solid rgba(255,255,255,0.07);
  border-radius: 16px;
  padding: 1.25rem 1.5rem;
  margin-bottom: 0.75rem;
  transition: border-color 0.2s;
  cursor: pointer;
}
```

```

.history-card:hover {
  border-color: rgba(59,130,246,0.3);
}
.history-card.fraud { border-left: 3px solid #EF4444; }
.history-card.legit { border-left: 3px solid #10B981; }
.history-time {
  font-family: 'DM Mono', monospace;
  font-size: 0.72rem;
  color: #475569;
}
.history-badge-fraud {
  display: inline-block;
  background: rgba(239,68,68,0.15);
  color: #FCA5A5;
  border: 1px solid rgba(239,68,68,0.3);
  font-size: 0.72rem;
  font-weight: 600;
  padding: 0.15rem 0.7rem;
  border-radius: 999px;
}
.history-badge-legit {
  display: inline-block;
  background: rgba(16,185,129,0.15);
  color: #6EE7B7;
  border: 1px solid rgba(16,185,129,0.3);
  font-size: 0.72rem;
  font-weight: 600;
  padding: 0.15rem 0.7rem;
  border-radius: 999px;
}
.history-preview {
  color: #94A3B8;
  font-size: 0.875rem;
  margin-top: 0.5rem;
}

```

```

    line-height: 1.5;
}
.history-conf {
    font-family: 'DM Mono', monospace;
    font-size: 0.82rem;
    color: #64748B;
}
.placeholder-wrap {
    text-align: center;
    padding: 5rem 2rem;
    color: #334155;
}
.placeholder-icon { font-size: 3.5rem; margin-bottom: 1rem; opacity: 0.5; }
.placeholder-txt { font-size: 1rem; }
.disclaimer {
    background: rgba(59,130,246,0.06);
    border: 1px solid rgba(59,130,246,0.15);
    border-radius: 14px;
    padding: 1.2rem 1.5rem;
    color: #64748B;
    font-size: 0.82rem;
    line-height: 1.7;
    text-align: center;
    margin-top: 2rem;
}
.stCaption, small { color: #475569 !important; }
p, span, div, label, li { color: #E2E8F0 !important; }
strong, b { color: #F1F5F9 !important; }
.stMarkdown p { color: #CBD5E1 !important; }
#MainMenu, footer, header { visibility: hidden; }
.stAlert { border-radius: 12px !important; }
[data-testid="stMetricValue"] { color: #fff !important; font-weight: 700 !important; }
[data-testid="stMetricLabel"] { color: #64748B !important; }
.streamlit-expanderHeader {

```

```

background: rgba(15,23,42,0.7) !important;
border-radius: 10px !important;
border: 1px solid rgba(255,255,255,0.07) !important;
color: #CBD5E1 !important;
font-weight: 500 !important;
font-size: 0.875rem !important;
}
.streamlit-expanderContent {
background: rgba(7,11,20,0.6) !important;
border: 1px solid rgba(255,255,255,0.05) !important;
border-top: none !important;
border-radius: 0 0 10px 10px !important;
}
[data-testid="stFileUploader"] {
background: rgba(15,23,42,0.7) !important;
border: 1.5px dashed rgba(59,130,246,0.35) !important;
border-radius: 14px !important;
}
[data-testid="stFileUploaderDropzone"] button {
background: linear-gradient(135deg, #3B82F6, #6D28D9) !important;
color: white !important;
border: none !important;
border-radius: 8px !important;
font-weight: 600 !important;
}
::-webkit-scrollbar { width: 6px; }
::-webkit-scrollbar-track { background: rgba(7,11,20,0.5); }
::-webkit-scrollbar-thumb { background: rgba(59,130,246,0.3); border-radius: 3px; }
</style>
"", unsafe_allow_html=True)
def render_verdict(result):
pred = result.get("prediction", {})
is_fraud = pred.get("is_fraudulent", False)
conf = pred.get("confidence", 0)

```

```

probs = pred.get("probabilities", {})
fraud_signals = pred.get("fraud_signals", [])
bert_raw = pred.get("bert_raw", probs)
if is_fraud:
    st.markdown(f"""
    <div class="verdict-fraud">
        <div class="verdict-icon">⚠️ </div>
        <div class="verdict-label">High Risk Detected</div>
        <div class="verdict-sub">Proceed with caution — verify through official channels</div>
        <div class="verdict-score">{probs.get('fraudulent', conf)}% Fraud Risk</div>
    </div>
    """, unsafe_allow_html=True)
else:
    st.markdown(f"""
    <div class="verdict-safe">
        <div class="verdict-icon">✓</div>
        <div class="verdict-label">Appears Legitimate</div>
        <div class="verdict-sub">Always independently verify company details</div>
        <div class="verdict-score">{probs.get('legitimate', conf)}% Safe Score</div>
    </div>
    """, unsafe_allow_html=True)
# Mini stats row
bert_fraud = bert_raw.get("fraudulent", probs.get("fraudulent", 0))
c1, c2, c3 = st.columns(3)
with c1:
    clr = "#EF4444" if is_fraud else "#10B981"
    st.markdown(f"""
    <div class="mini-stat-box">
        <div class="mini-stat-val" style="color: {clr}">{probs.get('fraudulent', 0)}%</div>
        <div class="mini-stat-lbl">Final Risk</div>
    </div>""", unsafe_allow_html=True)
with c2:
    clr2 = "#EF4444" if bert_fraud > 50 else "#10B981"
    st.markdown(f"""

```

```

<div class="mini-stat-box">
    <div class="mini-stat-val" style="color: {clr2}">{bert_fraud}%</div>
    <div class="mini-stat-lbl">BERT Score</div>
</div>""", unsafe_allow_html=True)
with c3:
    n = len(fraud_signals)
    clr3 = "#F59E0B" if n > 0 else "#10B981"
    st.markdown(f"""
<div class="mini-stat-box">
    <div class="mini-stat-val" style="color: {clr3}">{n}</div>
    <div class="mini-stat-lbl">Red Flags</div>
</div>""", unsafe_allow_html=True)
# Signal pills
if fraud_signals:
    st.markdown("<div style='margin-top:1rem'>", unsafe_allow_html=True)
    pills = "".join(f"<span class='signal-pill'>▶ {s}</span>" for s in fraud_signals)
    st.markdown(f"<div style='margin-top:0.75rem'>{pills}</div>", unsafe_allow_html=True)
    st.markdown("</div>", unsafe_allow_html=True)
def render_explanation(data):
    explanation = data.get("explanation", {})
    detailed_analysis = data.get("detailed_analysis", {})
    prediction = data.get("prediction", {})
    tab1, tab2, tab3 = st.tabs(["🔍 Advisory", "📊 Key Factors", "📄 Summary"])
    with tab1:
        risk_summary = detailed_analysis.get("risk_summary", {})
        overall_risk = risk_summary.get("overall_risk", "UNKNOWN")
        overall_advice = risk_summary.get("overall_advice", "Analysis complete.")
        risk_colors = {"CRITICAL": "#DC2626", "HIGH": "#EA580C", "MEDIUM": "#CA8A04",
"LOW": "#16A34A"}
        risk_icons = {"CRITICAL": "🔴", "HIGH": "🟠", "MEDIUM": "🟡", "LOW": "🟢"}
        color = risk_colors.get(overall_risk, "#6B7280")
        icon = risk_icons.get(overall_risk, "⬤")
        st.markdown(f"""

```

```

<div style="background:{color}18; border:1.5px solid {color}50; border-radius:12px;
padding:1.25rem; margin-bottom:1.25rem;">
    <div style="font-family:'Syne',sans-serif; font-size:1.1rem; font-weight:700; color:{color};
margin-bottom:0.5rem">{icon} Risk Level: {overall_risk}</div>
        <div style="color:#CBD5E1; font-size:0.875rem; line-
height:1.65">{overall_advice}</div>
    </div>
    "", unsafe_allow_html=True)
    left, right = st.columns([3, 2])
    with left:
        st.markdown('<div class="section-header">Risk Findings</div>', unsafe_allow_html=True)
        advisories = detailed_analysis.get("detailed_advisories", [])
        if advisories:
            for adv in advisories:
                lv = adv.get("risk_level", "Info")
                lv_clr = {"Critical": "#EF4444", "High": "#F97316", "Medium": "#EAB308", "Low":
"#22C55E"}.get(lv, "#3B82F6")
                with st.expander(f'{adv.get("icon",' ⚠️')} {adv.get("category",")}:
{adv.get("finding",")}):
                    st.markdown(f"""
                        <span style="background:{lv_clr}; color:white; padding:2px 10px; border-
radius:4px; font-size:11px; font-weight:700">{lv}</span>
                        <div style="margin-top:10px; color:#CBD5E1; line-height:1.7; font-
size:0.875rem">{adv.get('advisory',")}</div>
                    """, unsafe_allow_html=True)
            else:
                st.success("No specific red flags detected.")
    with right:
        st.markdown('<div class="section-header">Extracted Info</div>', unsafe_allow_html=True)
        info = detailed_analysis.get("extracted_info", {})
        if info.get("emails"):
            emails = info["emails"]
            is_personal = any("@gmail" in e or "@yahoo" in e or "@hotmail" in e for e in emails)
            clr = "#EF4444" if is_personal else "#22C55E"

```

```

        st.markdown(f'📧 **Email:** <span style=color:{clr}>{' .join(emails)}</span>',
unsafe_allow_html=True)
    if info.get("phone_numbers"):
        st.markdown(f'☎️ **Phone:** {' .join(info['phone_numbers'])}')
    if info.get("company_mentions"):
        st.markdown(f'🏢 **Company:** {' .join(info['company_mentions'][:2])}')
    if info.get("work_arrangement"):
        st.markdown(f'📅 **Work:** {info['work_arrangement'].replace('_', ' ').title()}')
    if info.get("messaging_apps"):
        st.markdown("💬 **Messaging:** <span style=color:#F97316>WhatsApp/Telegram
⚠️</span>", unsafe_allow_html=True)
    if info.get("requests_sensitive_info"):
        st.markdown("🔒 **Info:** <span style=color:#EF4444>SSN/Bank details requested
⚠️</span>", unsafe_allow_html=True)
        if not any([info.get("emails"), info.get("phone_numbers"),
info.get("company_mentions")]):
            st.info("No contact details extracted.")
    with tab2:
        chart_data = explanation.get("chart_data", {})
        words = chart_data.get("words", [])
        weights = chart_data.get("weights", [])
        if words and weights:
            try:
                colors = ["#EF4444" if w > 0 else "#10B981" for w in weights]
                fig = go.Figure(go.Bar(
                    x=weights, y=words, orientation="h",
                    marker=dict(color=colors, opacity=0.85),
                ))
                fig.update_layout(
                    title=dict(text="Word Impact on Prediction", font=dict(color="#F1F5F9", size=15,
family="Syne")),
                    xaxis=dict(title="Impact Score", title_font=dict(color="#94A3B8"),
tickfont=dict(color="#64748B", gridcolor="rgba(255,255,255,0.05)"),

```

```

        yaxis=dict(tickfont=dict(color="#94A3B8")),
        height=360,
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",
        font=dict(color="#94A3B8", family="DM Sans"),
        margin=dict(l=10, r=10, t=40, b=10),
    )
    st.plotly_chart(fig, use_container_width=True)
except Exception:
    st.info("Chart unavailable.")
else:
    signals = prediction.get("fraud_signals", [])
    if signals:
        for s in signals:
            st.markdown(f"▶ {s}")
    else:
        st.info("No key factor data available.")
with tab3:
    interpretation = explanation.get("interpretation", "")
    probs = prediction.get("probabilities", {})
    bert_raw = prediction.get("bert_raw", probs)
    final_risk = probs.get("fraudulent", 0)
    bert_risk = bert_raw.get("fraudulent", final_risk)
    signals = prediction.get("fraud_signals", [])
    is_high = final_risk > 50
    sc = "#DC2626" if is_high else "#16A34A"
    lbl = "HIGH RISK" if is_high else "LOW RISK"
    ico = "⚠" if is_high else "✓"

    st.markdown(f"""
    <div style="background:linear-gradient(135deg,{sc} 15,{sc} 08); border:1.5px solid {sc} 50;
border-radius:16px; padding:1.5rem; text-align:center; margin-bottom:1.25rem;">
        <div style="font-size:2.2rem; margin-bottom:0.4rem">{ico}</div>

```

```

        <div style="font-family:'Syne',sans-serif; font-size:1.5rem; font-weight:800; color:{sc};
margin-bottom:0.3rem">{lbl}</div>

        <div style="font-family:'DM Mono',monospace; font-size:2rem; font-weight:500;
color:#fff; margin-bottom:0.75rem">Final: {final_risk}%</div>

        <div style="display:flex; justify-content:center; gap:2.5rem;">
            <div><div style="color:#64748B; font-size:0.8rem">Rule Detection</div>
                <div style="color:{'#EF4444' if len(signals)>0 else '#10B981'}; font-size:1rem; font-
weight:700">{len(signals)} warning(s)</div></div>
                <div><div style="color:#64748B; font-size:0.8rem">BERT Score</div>
                    <div style="color:{'#EF4444' if bert_risk>50 else '#10B981'}; font-size:1rem; font-
weight:700">{bert_risk}% risk</div></div>
            </div>
        </div>

        """', unsafe_allow_html=True)
    if interpretation:
        st.markdown(f"""
            <div style="background:rgba(59,130,246,0.08); border-left:3px solid #3B82F6;
padding:1.2rem 1.5rem; border-radius:0 12px 12px 0; color:#CBD5E1; font-size:0.9rem; line-
height:1.75">
                {interpretation}
            </div>""", unsafe_allow_html=True)
    c1, c2 = st.columns(2)
    with c1:
        st.markdown('<div class="section-header" style="margin-top:1rem">Suspicious
Indicators</div>', unsafe_allow_html=True)
        pos = explanation.get("positive_features", [])
        if pos:
            for f in pos[:5]:
                w = f.get("word", str(f)) if isinstance(f, dict) else str(f)
                st.warning(w)
        elif signals:
            for s in signals[:5]:
                st.warning(s[:35] + ("..." if len(s) > 35 else ""))
        else:

```

```

        st.caption("None identified")
    with c2:
        st.markdown('<div class="section-header" style="margin-top:1rem">Legitimate
Indicators</div>', unsafe_allow_html=True)
        neg = explanation.get("negative_features", [])
        if neg:
            for f in neg[:5]:
                w = f.get("word", str(f)) if isinstance(f, dict) else str(f)
                st.success(w)
            else:
                st.caption("None identified")
def page_analyze(is_connected):
    col_in, col_out = st.columns(2, gap="large")
    with col_in:
        st.markdown('<div class="section-header">Input</div>', unsafe_allow_html=True)
        st.markdown('<div class="section-title">Paste Job Posting</div>', unsafe_allow_html=True)
        job_text = st.text_area(
            label="text",
            height=300,
            placeholder="Paste the complete job posting text here...\n\nInclude title, description,
requirements, contact info, etc.",
            label_visibility="collapsed",
            key="job_input"
        )
        st.caption(f'{len(job_text):,} characters')
    b1, b2 = st.columns(2)
    with b1:
        quick = st.button(" ⚡ Quick Analysis", use_container_width=True, disabled=not
is_connected)
    with b2:
        detailed = st.button(" 🔗 Detailed Analysis", use_container_width=True, disabled=not
is_connected)
    if quick and job_text:
        with st.spinner("Analyzing posting..."):

```

```

res = predict_job(job_text)
if res and res.get("success"):
    st.session_state.analysis_result = res
    st.session_state.analysis_type = "quick"
    add_to_history(job_text, res, "quick")
if detailed and job_text:
    with st.spinner("Running detailed analysis..."):
        res = get_explanation(job_text)
        if res and res.get("success"):
            st.session_state.analysis_result = res
            st.session_state.analysis_type = "detailed"
            add_to_history(job_text, res, "detailed")
if (quick or detailed) and not job_text:
    st.warning("Please paste a job posting to analyze.")
# Batch mode
with st.expander("📦 Batch Mode — Analyze Multiple Postings"):
    st.markdown("""<p style='color:#64748B; font-size:0.85rem; margin:0 0 0.75rem'>
        Separate postings with <code style='background:rgba(59,130,246,0.15); padding:1px 6px;
border-radius:4px; color:#93C5FD'>---</code> on its own line.</p>""",
    unsafe_allow_html=True)
    if st.button("Run Batch Analysis", use_container_width=True, disabled=not is_connected,
    key="batch_btn"):
        if job_text:
            postings = [p.strip() for p in job_text.split("---") if p.strip() and len(p.strip()) > 10]
            if not postings:
                st.warning("No valid postings found.")
            elif len(postings) == 1:
                st.info("Only 1 posting detected. Use Quick Analysis instead.")
            elif len(postings) > 100:
                st.error("Max 100 postings per batch.")
            else:
                with st.spinner(f"Analyzing {len(postings)} postings..."):
                    res = batch_predict(postings)
                    if res and res.get("success"):

```

```

st.success(f"Analyzed {res.get('total', len(postings))} postings")
for pred in res.get("predictions", []):
    idx = pred.get("index", 0) + 1
    lbl = pred.get("label", "Unknown")
    cf = pred.get("confidence", 0)
    ico = "☑" if lbl == "Legitimate" else "⚠"
    clr = "#10B981" if lbl == "Legitimate" else "#EF4444"
    st.markdown(f"**{ico} Posting {idx}:** <span style='color:{clr}'>{lbl}</span>
({cf}%)", unsafe_allow_html=True)
# Image upload
if IMAGE_SUPPORT:
    st.markdown("""<p style='color:#64748B; font-size:0.85rem; margin:0.75rem 0 0.5rem'>
 <strong style='color:#94A3B8'>Image Analysis</strong> — Upload a screenshot of a
job posting</p>""", unsafe_allow_html=True)
    uploaded = st.file_uploader("Upload image", type=["png", "jpg", "jpeg"],
label_visibility="collapsed", key="ocr_upload")
    if uploaded:
        img = Image.open(uploaded)
        st.image(img, caption="Uploaded Image", use_column_width=True)
        ia, ib = st.columns(2)
        with ia:
            iq = st.button("⚡ Quick", use_container_width=True, disabled=not is_connected,
key="img_q")
            with ib:
                idet = st.button("🔍 Detailed", use_container_width=True, disabled=not is_connected,
key="img_d")
            if iq or idet:
                uploaded.seek(0)
                with st.spinner("Extracting text and analyzing..."):
                    res = explain_image_api(uploaded.read())
                    if res and res.get("success"):
                        meta = res.get("metadata", {})
                        st.success(f"☑ Extracted {meta.get('text_length', 0)} characters")

```

```

        st.text_area("Extracted Text", meta.get("extracted_text", ""), height=120,
disabled=True)

        st.session_state.analysis_result = res
        st.session_state.analysis_type = "detailed" if idet else "quick"
        add_to_history(meta.get("extracted_text", "image upload"), res,
st.session_state.analysis_type)

        st.rerun()

with col_out:
    st.markdown('<div class="section-header">Results</div>', unsafe_allow_html=True)
    st.markdown('<div class="section-title">Analysis Output</div>', unsafe_allow_html=True)

    if st.session_state.get("analysis_result"):
        render_verdict(st.session_state.analysis_result)
        if st.session_state.get("analysis_type") == "detailed":
            render_explanation(st.session_state.analysis_result)
    else:
        st.markdown("""
<div class="placeholder-wrap">
    <div class="placeholder-icon">  </div>
    <div class="placeholder-txt">Paste a job posting and click Analyze<br>to get your risk
assessment</div>
</div>""", unsafe_allow_html=True)

def page_history():
    history = load_history()
    st.markdown('<div class="section-header">Past Analyses</div>', unsafe_allow_html=True)
    st.markdown('<div class="section-title">Analysis History</div>', unsafe_allow_html=True)
    if not history:
        st.markdown("""
<div class="placeholder-wrap">
    <div class="placeholder-icon">  </div>
    <div class="placeholder-txt">No analyses yet. Run an analysis to see history here.</div>
</div>""", unsafe_allow_html=True)
    return

# Stats row

```

```

total = len(history)
fraud_count = sum(1 for h in history if h.get("is_fraudulent"))
legit_count = total - fraud_count
avg_conf = sum(h.get("confidence", 0) for h in history) / total if total else 0
sc1, sc2, sc3, sc4 = st.columns(4)
with sc1:
    st.markdown(f"""<div class="mini-stat-box"><div class="mini-stat-val">{total}</div><div
class="mini-stat-lbl">Total Analyses</div></div>""", unsafe_allow_html=True)
with sc2:
    st.markdown(f"""<div class="mini-stat-box"><div class="mini-stat-val"
style="color:#EF4444">{fraud_count}</div><div class="mini-stat-
lbl">Flagged</div></div>""", unsafe_allow_html=True)
with sc3:
    st.markdown(f"""<div class="mini-stat-box"><div class="mini-stat-val"
style="color:#10B981">{legit_count}</div><div class="mini-stat-lbl">Safe</div></div>""",
unsafe_allow_html=True)
with sc4:
    st.markdown(f"""<div class="mini-stat-box"><div class="mini-stat-
val">{avg_conf:.0f}%</div><div class="mini-stat-lbl">Avg Confidence</div></div>""",
unsafe_allow_html=True)
st.markdown("<div style='height:1.5rem'></div>", unsafe_allow_html=True)
# Filter bar
fc1, fc2, fc3 = st.columns([2, 1, 1])
with fc1:
    search = st.text_input("🔍 Search history", placeholder="Filter by keywords...",
label_visibility="collapsed")
with fc2:
    filter_type = st.selectbox("Filter", ["All", "Fraudulent", "Legitimate"],
label_visibility="collapsed")
with fc3:
    if st.button("🗑️ Clear All History", use_container_width=True):
        clear_history()
        st.success("History cleared!")
        st.rerun()

```

```

# Filter
filtered = history
if filter_type == "Fraudulent":
    filtered = [h for h in filtered if h.get("is_fraudulent")]
elif filter_type == "Legitimate":
    filtered = [h for h in filtered if not h.get("is_fraudulent")]
if search:
    sq = search.lower()
    filtered = [h for h in filtered if sq in h.get("text_preview", "").lower() or sq in h.get("label",
""").lower()]
st.caption(f"Showing {len(filtered)} of {total} entries")
st.markdown("<div style='height:0.5rem'></div>", unsafe_allow_html=True)
# Entry list
for i, entry in enumerate(filtered):
    is_fraud = entry.get("is_fraudulent", False)
    card_class = "fraud" if is_fraud else "legit"
    badge = f'<span class="history-badge-fraud">⚠️ Fraudulent</span>' if is_fraud else f'<span
class="history-badge-legit">✓ Legitimate</span>'
    conf = entry.get("confidence", 0)
    atype = entry.get("analysis_type", "quick").title()
    timestamp = entry.get("timestamp", "")
    preview = entry.get("text_preview", "")
    signals = entry.get("fraud_signals", [])
    with st.expander(f'{'🚩' if is_fraud else '✅'} {preview[:60]}...', expanded=False):
        st.markdown(f'"""
            <div style="display:flex; align-items:center; justify-content:space-between; margin-
bottom:0.75rem; flex-wrap:wrap; gap:0.5rem;">
                <div style="display:flex; align-items:center; gap:0.75rem">
                    {badge}
                    <span style="background:rgba(59,130,246,0.15); color:#93C5FD; border:1px solid
rgba(59,130,246,0.3); font-size:0.72rem; font-weight:600; padding:0.15rem 0.7rem; border-
radius:999px">{atype}</span>
                </div>

```

```

<div style="display:flex; align-items:center; gap:1.25rem">
    <span class="history-conf">Confidence: {conf}%</span>
    <span class="history-time">{timestamp}</span>
</div>
</div>
""", unsafe_allow_html=True)
probs = entry.get("probabilities", {})
p1, p2 = st.columns(2)
with p1:
    st.markdown(f"***Fraud Risk:** <span style='color:#EF4444; font-family:DM
Mono,monospace'>{probs.get('fraudulent', 0)}%</span>", unsafe_allow_html=True)
with p2:
    st.markdown(f"***Safe Score:** <span style='color:#10B981; font-family:DM
Mono,monospace'>{probs.get('legitimate', 0)}%</span>", unsafe_allow_html=True)
if signals:
    st.markdown("***Red Flags:**")
    pills = "".join(f"<span class='signal-pill'>▶ {s}</span>" for s in signals)
    st.markdown(f"<div>{pills}</div>", unsafe_allow_html=True)
full_text = entry.get("full_text", "")
if full_text:
    st.markdown("***Full Text:**")
    st.text_area("", full_text, height=160, disabled=True, key=f"hist_text_{entry.get('id',
i)}}")
# Re-analyze button
if st.button(f"🔄 Re-Analyze", key=f"reanalyze_{entry.get('id', i)}"):
    st.session_state.job_input = full_text
    st.session_state.page = "analyze"
    st.rerun()
def main():
    load_css()
    if "analysis_result" not in st.session_state:
        st.session_state.analysis_result = None
    if "analysis_type" not in st.session_state:
        st.session_state.analysis_type = None

```

```

if "page" not in st.session_state:
    st.session_state.page = "analyze"
is_connected = check_api()
dot_class = "dot-green" if is_connected else "dot-red"
status_txt = "API Connected" if is_connected else "API Disconnected"
history = load_history()
hist_count = len(history)
st.markdown(f"""
<div class="nav-bar">
    <div class="nav-logo">
        <div class="nav-logo-icon">  </div>
        JobGuard AI
    </div>
    <div class="nav-status">
        <span class="nav-status-dot {dot_class}"></span>{status_txt}
    </div>
</div>
""", unsafe_allow_html=True)
p1, p2, p3 = st.columns([1, 1, 8])
with p1:
    if st.button(f"  Analyze", use_container_width=True):
        st.session_state.page = "analyze"
with p2:
    if st.button(f"  History ({hist_count})", use_container_width=True):
        st.session_state.page = "history"
st.markdown("<div style='height:0.25rem; border-bottom:1px solid rgba(255,255,255,0.06); margin-bottom:2rem'></div>", unsafe_allow_html=True)
if st.session_state.page == "analyze":
    st.markdown("""
<div class="hero-section">
    <div class="hero-grid"></div>
    <div class="hero-badge">AI-Powered Detection System</div>
    <h1 class="hero-title">Detect Fake Job<br><span>Postings Instantly</span></h1>

```

```
<p class="hero-sub">Hybrid BERT + Rule-Based analysis trained on 17,000+ job postings
to protect you from recruitment scams.</p>
```

```
<div class="hero-stats">
    <div><div class="hero-stat-num">97%</div><div class="hero-stat-
label">Accuracy</div></div>
```

```
    <div><div class="hero-stat-num">8</div><div class="hero-stat-label">Scam
Types</div></div>
```

```
    <div><div class="hero-stat-num">< 2s</div><div class="hero-stat-label">Analysis
Time</div></div>
```

```
</div>
```

```
</div>
```

```
""", unsafe_allow_html=True)
```

```
if st.session_state.page == "analyze":
```

```
    page_analyze(is_connected)
```

```
elif st.session_state.page == "history":
```

```
    page_history()
```

```
st.markdown("""
```

```
<div class="disclaimer">
```

```
    <strong style='color:#93C5FD'>⚠ Disclaimer</strong><br>
```

```
    JobGuard AI provides risk assessment guidance only — not definitive fraud detection.
```

```
    Always verify job postings independently through official company websites and trusted
review platforms.
```

```
</div>
```

```
""", unsafe_allow_html=True)
```

```
if __name__ == "__main__":
```

```
    main()
```

2. Backend

preprocessing.py

```
import sys
import re
import time
import logging
import traceback
from pathlib import Path
from functools import wraps
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from flask import Flask, request, jsonify
from flask_cors import CORS
from src.model import JobFraudDetector
from config.settings import FLASK_CONFIG, MODEL_DIR, LOGGING_CONFIG, LABEL_MAP
logging.basicConfig(
    level=getattr(logging, LOGGING_CONFIG["level"]),
    format=LOGGING_CONFIG["format"]
)
logger = logging.getLogger(__name__)
app = Flask(__name__)
CORS(app, resources={r"/api/*": {"origins": "*"}})
detector = None
def clean_text(text: str) -> str:
    if not text:
        return ""
    text = str(text)
    text = re.sub(r'<[^>]+>', '', text)
    text = re.sub(r'http\S+|www\.\S+', '', text)
    text = re.sub(r'\s+', '', text).strip()
    return text
def initialize_services():
    global detector
```

```

logger.info("Initializing ML services...")
detector = JobFraudDetector()
model_path = MODEL_DIR / "best_model.pt"
if model_path.exists():
    try:
        detector.load_model(model_path)
        logger.info("Successfully loaded BERT model")
    except Exception as error:
        logger.warning(f"Failed to load model: {error}. Using demo mode.")
        detector.initialize_model()
else:
    logger.info("No trained model found. Using demo mode.")
    detector.initialize_model()
def handle_errors(function):
    @wraps(function)
    def decorated_function(*args, **kwargs):
        try:
            return function(*args, **kwargs)
        except Exception as error:
            logger.error(f"Error in {function.__name__}: {error}\n{traceback.format_exc()}")
            return jsonify({
                'success': False,
                'error': str(error)
            }), 500
    return decorated_function
FRAUD_KEYWORDS = {
    'send ssn': ('Requests SSN', 5),
    'bank details': ('Requests banking info', 5),
    'credit card': ('Requests credit card', 5),
    'wire money': ('Requests money transfer', 5),
    'processing fee': ('Upfront fees required', 4.5),
    'gift card': ('Gift card scam indicator', 4.5),
    'earn money fast': ('Quick money promise', 4),
    '$5000': ('Unrealistic salary claims', 4),

```

```

'easy money': ('Easy money claims', 3.5),
'guaranteed': ('Guarantee claims', 2.5),
'no interview': ('No interview required', 4),
'no experience': ('No experience required', 3),
'urgent': ('Urgency tactics', 2.5),
'!!!': ('Excessive punctuation', 2),
'telegram': ('Informal contact method', 2),
'gmail.com': ('Free email service', 2.5),
'work from home': ('WFH (can be misused)', 1.5),
}

LEGITIMATE_KEYWORDS = {
    'experience required': ('Experience requirements', 4),
    'bachelor': ('Education requirements', 4),
    'degree': ('Degree mentioned', 2.5),
    'qualifications': ('Qualification requirements', 3),
    'health insurance': ('Standard benefits', 4.5),
    '401k': ('Retirement benefits', 4.5),
    'pto': ('Paid time off', 4),
    'competitive salary': ('Professional language', 3),
    'interview': ('Interview process', 3),
    'resume': ('Resume required', 2),
}

def generate_explanation(text: str, prediction: dict) -> dict:
    text_lower = text.lower()
    fraud_features = []
    legit_features = []
    for keyword, (description, severity) in FRAUD_KEYWORDS.items():
        if keyword in text_lower:
            # Calculate weight (positive = toward fraud)
            weight = round((0.1 + severity / 5 * 0.4), 3)
            fraud_features.append({
                'word': keyword,
                'weight': weight,
                'description': description

```

```

    })
for keyword, (description, severity) in LEGITIMATE_KEYWORDS.items():
    if keyword in text_lower:
        # Calculate weight (negative = toward legitimate)
        weight = round(-(0.1 + severity / 5 * 0.4), 3)
        legit_features.append({
            'word': keyword,
            'weight': weight,
            'description': description
        })
fraud_features.sort(key=lambda x: -x['weight'])
legit_features.sort(key=lambda x: x['weight'])
is_fraud = prediction['prediction'] == 1
confidence = prediction['confidence']
if is_fraud:
    interpretation = (
        f"This job posting is classified as **Fraudulent** "
        f"with {confidence:.1f}% confidence.\n\n"
    )
    if fraud_features:
        interpretation += "***Suspicious indicators:**\n"
        for feature in fraud_features[:5]:
            interpretation += f"- '{feature['word']}': {feature['description']}\n"
        interpretation += (
            "\n\n**Recommendation:** Exercise caution. "
            "Verify the company through official channels."
        )
else:
    interpretation = (
        f"This job posting appears **Legitimate** "
        f"with {confidence:.1f}% confidence.\n\n"
    )
    if legit_features:
        interpretation += "***Legitimate indicators:**\n"

```

```

        for feature in legit_features[:5]:
            interpretation += f'- {feature['word']}: {feature['description']}\n'
        interpretation += (
            "\n\n**Recommendation:** While this appears legitimate, "
            "always verify the company."
        )
    all_features = fraud_features + legit_features
    chart_words = [feature['word'] for feature in all_features[:10]]
    chart_weights = [feature['weight'] for feature in all_features[:10]]
    chart_colors = [
        '#FF6B6B' if feature['weight'] > 0 else '#4ECDC4'
        for feature in all_features[:10]
    ]
    return {
        'feature_contributions': all_features[:10],
        'positive_features': fraud_features[:5],
        'negative_features': legit_features[:5],
        'interpretation': interpretation,
        'highlighted_text': text,
        'chart_data': {
            'words': chart_words,
            'weights': chart_weights,
            'colors': chart_colors
        }
    }

@app.route('/api/health', methods=['GET'])
def health_check():
    return jsonify({
        'status': 'healthy',
        'model_loaded': detector is not None,
        'timestamp': time.time()
    })

@app.route('/api/predict', methods=['POST'])
@handle_errors

```

```

def predict():
    data = request.get_json()
    if not data:
        return jsonify({
            'success': False,
            'error': 'No data provided'
        }), 400
    text = data.get('text')
    if not text:
        fields = ['title', 'company_profile', 'description', 'requirements', 'benefits']
        text = ''.join(str(data.get(field, "")) for field in fields)
    if len(text.strip()) < 10:
        return jsonify({
            'success': False,
            'error': 'Text too short (minimum 10 characters)'
        }), 400
    start_time = time.time()
    cleaned_text = clean_text(text)
    result = detector.predict(cleaned_text)
    inference_time_ms = round((time.time() - start_time) * 1000, 2)
    response_data = {
        'success': True,
        'prediction': {
            'label': result['label'],
            'confidence': result['confidence'],
            'probabilities': {
                'legitimate': result['probabilities']['legitimate'],
                'fraudulent': result['probabilities']['fraudulent']
            },
            'is_fraudulent': result['prediction'] == 1
        },
        'metadata': {
            'text_length': len(text),
            'inference_time_ms': inference_time_ms
        }
    }

```

```

    }
}
if 'fraud_signals' in result:
    response_data['prediction']['fraud_signals'] = result['fraud_signals']
return jsonify(response_data)
@app.route('/api/explain', methods=['POST'])
@handle_errors
def explain():
    data = request.get_json()
    if not data or 'text' not in data:
        return jsonify({
            'success': False,
            'error': 'No text provided'
        }), 400
    text = data['text']
    if len(text.strip()) < 10:
        return jsonify({
            'success': False,
            'error': 'Text too short (minimum 10 characters)'
        }), 400
    start_time = time.time()
    cleaned_text = clean_text(text)
    prediction = detector.predict(cleaned_text)
    explanation = generate_explanation(text, prediction)
    detailed_analysis = detector.extract_posting_details(text)
    explanation_time_ms = round((time.time() - start_time) * 1000, 2)
    response_data = {
        'success': True,
        'prediction': {
            'label': prediction['label'],
            'confidence': prediction['confidence'],
            'probabilities': {
                'legitimate': prediction['probabilities']['legitimate'],
                'fraudulent': prediction['probabilities']['fraudulent']
            }
        }
    }

```

```

        },
        'is_fraudulent': prediction['prediction'] == 1
    },
    'explanation': explanation,
    'detailed_analysis': detailed_analysis,
    'metadata': {
        'text_length': len(text),
        'explanation_time_ms': explanation_time_ms
    }
}

if 'fraud_signals' in prediction:
    response_data['prediction']['fraud_signals'] = prediction['fraud_signals']
return jsonify(response_data)

@app.route('/api/batch-predict', methods=['POST'])
@handle_errors
def batch_predict():
    data = request.get_json()
    if not data or 'texts' not in data:
        return jsonify({
            'success': False,
            'error': 'No texts provided'
        }), 400
    texts = data['texts']
    if not isinstance(texts, list):
        return jsonify({
            'success': False,
            'error': 'texts must be an array'
        }), 400
    if len(texts) == 0:
        return jsonify({
            'success': False,
            'error': 'texts array is empty'
        }), 400
    if len(texts) > 100:

```

```

    return jsonify({
        'success': False,
        'error': 'Maximum 100 texts per request'
    }), 400
predictions = []
for index, text in enumerate(texts):
    if len(text.strip()) < 10:
        # Text too short - add error entry
        predictions.append({
            'index': index,
            'error': 'Text too short'
        })
    else:
        # Make prediction
        cleaned_text = clean_text(text)
        result = detector.predict(cleaned_text)
        predictions.append({
            'index': index,
            'label': result['label'],
            'confidence': result['confidence'], # Already a percentage from model
            'is_fraudulent': result['prediction'] == 1
        })
return jsonify({
    'success': True,
    'predictions': predictions,
    'total': len(texts)
})

@app.route('/api/model-info', methods=['GET'])
def model_info():
    return jsonify({
        'success': True,
        'model': {
            'name': 'Fake Job Detector',
            'type': 'BERT-based classifier',

```

```

        'labels': ['Legitimate', 'Fraudulent']
    }
})
@app.route('/api/sample-jobs', methods=['GET'])
def get_sample_jobs():
    return jsonify({
        'success': True,
        'samples': {
            'legitimate': [{
                'title': 'Software Engineer',
                'description': (
                    "Bachelor's degree required. 3+ years experience. "
                    "Health insurance, 401k matching."
                )
            }],
            'suspicious': [{
                'title': 'EARN $5000/WEEK!!!',
                'description': (
                    'No experience needed! Send bank details! '
                    'Unlimited earning!'
                )
            }]
        }
    })
@app.route('/api/explain-image', methods=['POST'])
@handle_errors
def explain_image():
    import base64
    from io import BytesIO
    from PIL import Image
    from src.image_ocr import ImageOCR
    data = request.get_json()
    if not data or 'image' not in data:
        return jsonify({

```

```

        'success': False,
        'error': 'No image provided'
    }), 400
image_data = data['image']
try:
    if ',' in image_data:
        image_data = image_data.split(',')[1]
    image_bytes = base64.b64decode(image_data)
    pil_image = Image.open(BytesIO(image_bytes))
except Exception as e:
    return jsonify({
        'success': False,
        'error': f'Invalid image data: {str(e)}'
    }), 400
start_time = time.time()
try:
    ocr = ImageOCR()
    extracted_text = ocr.extract_text(pil_image)
except Exception as e:
    return jsonify({
        'success': False,
        'error': f'OCR failed: {str(e)}'
    }), 500
if not extracted_text or len(extracted_text.strip()) < 10:
    return jsonify({
        'success': False,
        'error': 'Could not extract enough text from image (minimum 10 characters)'
    }), 400
cleaned_text = clean_text(extracted_text)
prediction = detector.predict(cleaned_text)
explanation = generate_explanation(extracted_text, prediction)
detailed_analysis = detector.extract_posting_details(extracted_text)
processing_time_ms = round((time.time() - start_time) * 1000, 2)
response_data = {

```

```

'success': True,
'prediction': {
    'label': prediction['label'],
    'confidence': prediction['confidence'],
    'probabilities': {
        'legitimate': prediction['probabilities']['legitimate'],
        'fraudulent': prediction['probabilities']['fraudulent']
    },
    'is_fraudulent': prediction['prediction'] == 1
},
'explanation': explanation,
'detailed_analysis': detailed_analysis,
'metadata': {
    'source': 'image',
    'extracted_text': extracted_text,
    'text_length': len(extracted_text),
    'processing_time_ms': processing_time_ms
}
}
if 'fraud_signals' in prediction:
    response_data['prediction']['fraud_signals'] = prediction['fraud_signals']
return jsonify(response_data)

if __name__ == '__main__':
    initialize_services()
    app.run(
        host=FLASK_CONFIG["host"],
        port=FLASK_CONFIG["port"],
        debug=FLASK_CONFIG["debug"]
    )

```

5.2 Model Training and Implementation

The model training phase is an essential part of the system, where a deep learning model is trained to classify job postings as fraudulent or legitimate. In this project, a BERT-based model is trained using the EMSCAD dataset, which contains labelled job postings. The training process includes data preparation, preprocessing, tokenization, handling class imbalance, model training, evaluation, and saving the trained model for future use.

1. Dataset Description

The dataset used for training is the EMSCAD Fake Job Postings Dataset, which contains approximately 17,000+ job listings labelled as fraudulent (1) or legitimate (0). The dataset includes multiple fields such as job title, company profile, description, requirements, and benefits. These fields are combined into a single text column for model training.

2. Data Preprocessing

The dataset is preprocessed before training to ensure better model performance. All text fields are combined into a single column, and missing values are handled by replacing them with empty strings. Short or irrelevant entries are removed to improve data quality. This step ensures that the model receives clean and meaningful input data.

3. Tokenization

The cleaned text is tokenized using the BERT tokenizer. Tokenization converts text into numerical format by splitting it into tokens and mapping them to corresponding IDs. Padding and truncation are applied to ensure uniform input length, making it suitable for the BERT model.

4. Handling Imbalanced Data

The dataset contains more legitimate job postings than fraudulent ones, leading to class imbalance. To address this issue, SMOTE Tomek is applied, which combines SMOTE (Synthetic Minority Oversampling Technique) and Tomek Links. This technique generates synthetic fraud samples and removes noisy data, improving model accuracy and balance.

5. Class Weights

In addition to SMOTE, class weights are calculated to give more importance to the minority class (fraudulent jobs). This helps the model focus more on correctly identifying fraudulent postings and improves recall.

6. Training Process

The model training is performed using k-fold cross-validation (5 folds) to ensure robustness and reliability. In each fold, the dataset is split into training and validation sets. The model is trained

for multiple epochs using a low learning rate to improve performance. Early stopping is implemented to prevent overfitting, where training stops if validation loss does not improve.

7. Model Used

The system uses a BERT (Bidirectional Encoder Representations from Transformers) model for classification. The model is fine-tuned on the dataset to understand contextual information in job descriptions and accurately detect fraudulent patterns.

8. Model Saving

After training, the best-performing model is saved to the system. Checkpoints are also saved during training to preserve intermediate results. The final model is stored for use in prediction during runtime.

9. Evaluation Metrics

The model is evaluated using metrics such as:

- Accuracy
- Precision
- Recall
- F1-score

These metrics help measure the effectiveness of the model in detecting fraudulent job postings.

10. Logging

The training process includes logging mechanisms that record important details such as training progress, validation accuracy, and final metrics. These logs are stored in a JSON file for analysis and future improvements.

Pseudo Code

START

Load dataset

Prepare data (combine text fields)

Tokenize text using BERT tokenizer

Split data using k-fold cross validation

FOR each fold:

 Train model

 Validate model

 Apply early stopping

END FOR

Apply SMOTE to balance data

Calculate class weights
Evaluate model performance
Save best model
Store logs
END

5.3 Testing

The testing process of the Fake Job Detection System is a systematic approach used to ensure that the application functions correctly, efficiently, and reliably under different conditions. Since the system integrates multiple components such as preprocessing, a BERT-based model, rule-based logic, OCR, explainable AI, backend APIs, and a frontend interface, testing is performed at multiple levels to validate each module as well as the overall system. The goal of testing is to identify and eliminate errors, ensure accuracy in fraud detection, and verify that the system meets both functional and non-functional requirements.

➤ Testing Strategy

The testing strategy followed in this project is layered and comprehensive, covering:

Unit Testing (individual components)
Integration Testing (module interaction)
API Testing (backend endpoints)
End-to-End Testing (complete workflow)
Performance Testing (speed and efficiency)

This structured approach ensures that every part of the system is tested independently and in combination.

➤ Unit Testing

Unit testing focuses on verifying the correctness of individual modules in isolation. Each core component of the system is tested separately to ensure that it performs its intended function.

The preprocessing module is tested to confirm that text cleaning removes unwanted elements such as HTML tags, URLs, email addresses, and special characters. It also verifies that tokenization correctly converts text into a format suitable for the BERT model.

The model component is tested to ensure proper initialization and prediction functionality. The system verifies that predictions cannot be made unless the model is loaded, ensuring proper error handling. The explainability module (LIME) is tested to check whether it correctly generates explanations, highlights important words, and produces structured outputs such as feature contributions and interpretation.

These tests ensure that each individual component behaves correctly before integrating them into the full system.

➤ **Integration Testing**

Integration testing verifies that different modules of the system work together correctly. In this stage, the interaction between preprocessing, model prediction, rule-based logic, and explainability is tested.

The system processes sample job postings through the complete pipeline:

Input → Preprocessing → Model → Rule-based detection → Explanation

The output is validated to ensure that:

- Predictions are generated correctly
- Confidence scores are accurate
- Explanations are properly linked to predictions

This ensures that data flows smoothly between modules and that there are no compatibility issues.

➤ **API Testing**

The backend of the system is built using Flask, which provides REST API endpoints. API testing ensures that these endpoints function correctly and return valid responses.

- The /health endpoint is tested to verify that the system is running and the model is loaded successfully.
- The /predict endpoint is tested by sending job posting text and verifying that the response includes prediction results, labels, and confidence scores.
- The /explain endpoint is tested to ensure that it returns detailed explanations along with prediction results.
- The /batch-predict endpoint is tested to verify that multiple job postings can be processed simultaneously and that results are returned for each input.

API testing ensures proper communication between frontend and backend.

➤ **End-to-End (E2E) Testing**

End-to-end testing evaluates the entire system from the user's perspective. This testing simulates real-world usage scenarios to ensure that the system works as expected.

Using tools like Playwright, the following actions are tested:

- Entering job posting text in the interface
- Clicking analysis buttons (Quick Analysis, Detailed Analysis)
- Viewing prediction results and explanations
- Performing batch analysis

- Uploading images for OCR-based detection

The system is tested with different types of inputs such as legitimate jobs, obvious scams, and subtle fraud cases. This ensures that the system behaves correctly in real-world conditions.

❖ Testing Implementation

Sample Test Code (Unit Testing)

```
class TestDataPreprocessor:
    """Tests for DataPreprocessor class."""

    def setup_method(self):
        self.preprocessor = DataPreprocessor()

    def test_clean_text_basic(self):
        """Test basic text cleaning."""
        text = "Hello World!"
        cleaned = self.preprocessor.clean_text(text)
        assert cleaned == "hello world!"
```

The above test case checks whether the text cleaning function converts uppercase text into lowercase properly. It ensures that the preprocessing step works correctly.

Testing Modules

1. Data Preprocessing Testing:

```
def test_clean_text_html(self):
    """Test HTML tag removal."""
    text = "<p>Hello <b>World</b></p>"
    cleaned = self.preprocessor.clean_text(text)
    assert "<" not in cleaned
    assert ">" not in cleaned
```

This test verifies that HTML tags are removed from the input text.

It ensures that the preprocessing module cleans unnecessary formatting elements before passing data to the model.

2. Tokenization Testing

```
def test_tokenize(self):
    """Test tokenization."""
    text = "Software Engineer"
    tokens = self.preprocessor.tokenize(text)
    assert 'input_ids' in tokens
    assert 'attention_mask' in tokens
```

This test checks whether the tokenizer correctly converts text into numerical token IDs required by the BERT model.

3. Model Testing

```
def test_model_initialization(self):
    """Test model can be initialized."""
    model = BertClassifier()
    assert model is not None
    assert model.num_labels == 2
```

This test ensures that the BERT model is initialized properly with two output classes: fraudulent and legitimate.

4. Prediction Testing

```
def test_predict_without_model(self):
    """Test prediction fails without model."""
    with pytest.raises(ValueError):
        self.detector.predict("test text")
```

This test verifies that the system raises an error if prediction is attempted before loading the model, ensuring proper workflow enforcement.

5. LIME Explanation Testing

```
class TestLIMEExplainer:
    def test_highlight_text(self):
        highlighted = self.explainer.highlight_text(text, contributions)
        assert '<span>' in highlighted
```

This test ensures that important words in the text are highlighted correctly, helping users understand why a prediction was made.

6. Integration Testing

```
def test_end_to_end_sample_data(self):
    """Test full pipeline with sample data."""
    # Load and preprocess data
    preprocessor = DataPreprocessor()
    df = preprocessor.load_dataset()
    df = preprocessor.preprocess_dataset(df)

    # Verify data
    assert len(df) > 0
    assert df['fraudulent'].nunique() == 2

    # Initialize detector
    detector = JobFraudDetector()
    detector.initialize_model()

    # Make prediction
    sample_text = df['combined_text'].iloc[0]
    result = detector.predict(sample_text)

    assert 'prediction' in result
    assert 'label' in result
    assert 'confidence' in result
```

This test checks the complete system workflow from preprocessing to prediction. It ensures that all modules work together correctly.

Testing Summary

The testing of the system was carried out to ensure that all components work correctly and efficiently. Different types of testing, including unit testing, integration testing, API testing, and end-to-end testing, were performed. The preprocessing module was tested to confirm that text cleaning, tokenization, and data handling are working properly. The model was tested with various job postings, including legitimate jobs, obvious fraud cases, and subtle scam examples, to verify its accuracy.

The backend APIs were tested to ensure correct responses for prediction, explanation, and batch processing. The frontend interface was also tested to confirm that user inputs, buttons, and output display functions work as expected. Integration testing ensured that all modules, including preprocessing, model prediction, and explanation, work together smoothly.

The results show that the system successfully detects fraudulent job postings and provides accurate predictions along with explanations. All test cases passed successfully, and the system performs efficiently for both single and multiple inputs. Overall, the testing confirms that the system is reliable, accurate, and ready for real-world usage.

➤ Test Case Design

Test cases are designed to cover various scenarios, including:

1. Legitimate Job Postings

Expected result: Classified as legitimate

Purpose: Verify that the system does not falsely flag genuine jobs

2. Obvious Fraud Cases

Examples: “Earn ₹5000/day”, “No experience needed”, “Send bank details”

Expected result: Classified as fraudulent

Purpose: Ensure detection of clear scams

3. Subtle Fraud Cases

Examples: Professional-looking job postings with hidden fees

Expected result: Classified as fraudulent

Purpose: Test deep learning model capability

4. Edge Cases

Short or incomplete job descriptions

Mixed legitimate and suspicious content

Expected result: System handles gracefully.

➤ Performance Testing

Performance testing evaluates how quickly and efficiently the system processes inputs. The system is tested to ensure that:

- Predictions are generated within a few seconds
- Batch processing handles multiple inputs efficiently
- OCR processing does not significantly delay results

This ensures a smooth user experience.

➤ Error Handling Testing

The system is tested for robustness by providing invalid or unexpected inputs such as:- Empty text, Incorrect image formats, API failures

The system should handle these gracefully by displaying appropriate error messages instead of crashing.

➤ Reliability Testing

Reliability testing ensures that the system produces consistent results over multiple runs. The same input is tested multiple times to verify that predictions remain stable and accurate.

Validation And Accuracy Testing The system is validated using labeled datasets to measure: Accuracy, Precision, Recall

This ensures that the model performs well in detecting fraudulent job postings.

How To Run The Code In Command Prompt

The project is executed using the Command Prompt by setting up the environment, installing dependencies, and running both backend and frontend services. The system follows a client-server architecture, where the backend is implemented using Flask and the frontend using Streamlit.

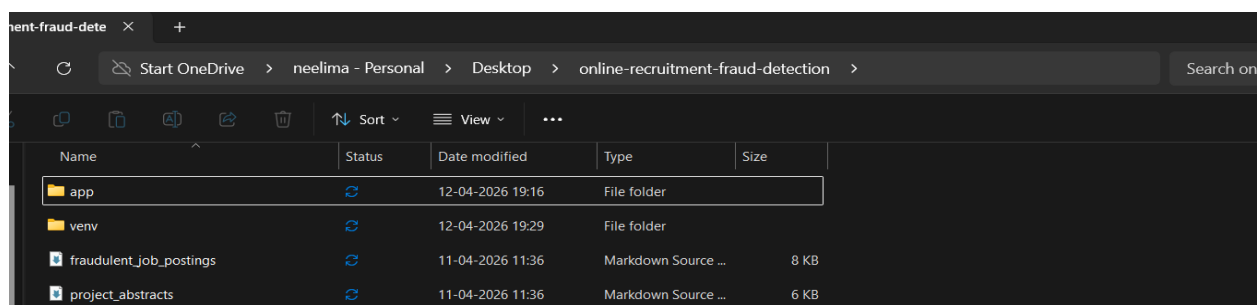
Step-1: Create a Project Folder

Create a folder for your project

Eg: online_recruitment_fraud_detection

Step-2: Place Project Files in Folder

Copy all your project files into the folder



Step-3: Open Command Prompt

Press:-

Win + R → cmd → Enter

Step-4: Navigate to Project Folder

Copy the folder path and paste it in the command prompt with adding cd.

```
Command Prompt
Microsoft Windows [Version 10.0.22631.6199]
(c) Microsoft Corporation. All rights reserved.

C:\Users\neelima>cd C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection
C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection>
```

Step-5: Create Virtual Environment

For creating virtual environment we use : Python -m venv venv

```
Command Prompt - python
Microsoft Windows [Version 10.0.22631.6199]
(c) Microsoft Corporation. All rights reserved.

C:\Users\neelima>cd C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection
C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection>python -m venv venv
```

Step-6: Activate Virtual Environment

```
Command Prompt
Microsoft Windows [Version 10.0.22631.6199]
(c) Microsoft Corporation. All rights reserved.

C:\Users\neelima>cd C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection
C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection>python -m venv venv
C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection>venv\Scripts\activate
(venv) C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection>
```

Step-7: Install Required Libraries

To install the requirement libraries we use : pip install -r requirements.txt

Step-8: Run Frontend Code

Move to Frontend folder:

Copy the path of folder in Frontend file and paste it in the command prompt with cd

```
(venv) C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection\app>cd C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection\app\frontend
(venv) C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection\app\frontend>streamlit run app.py

You can now view your Streamlit app in your browser.

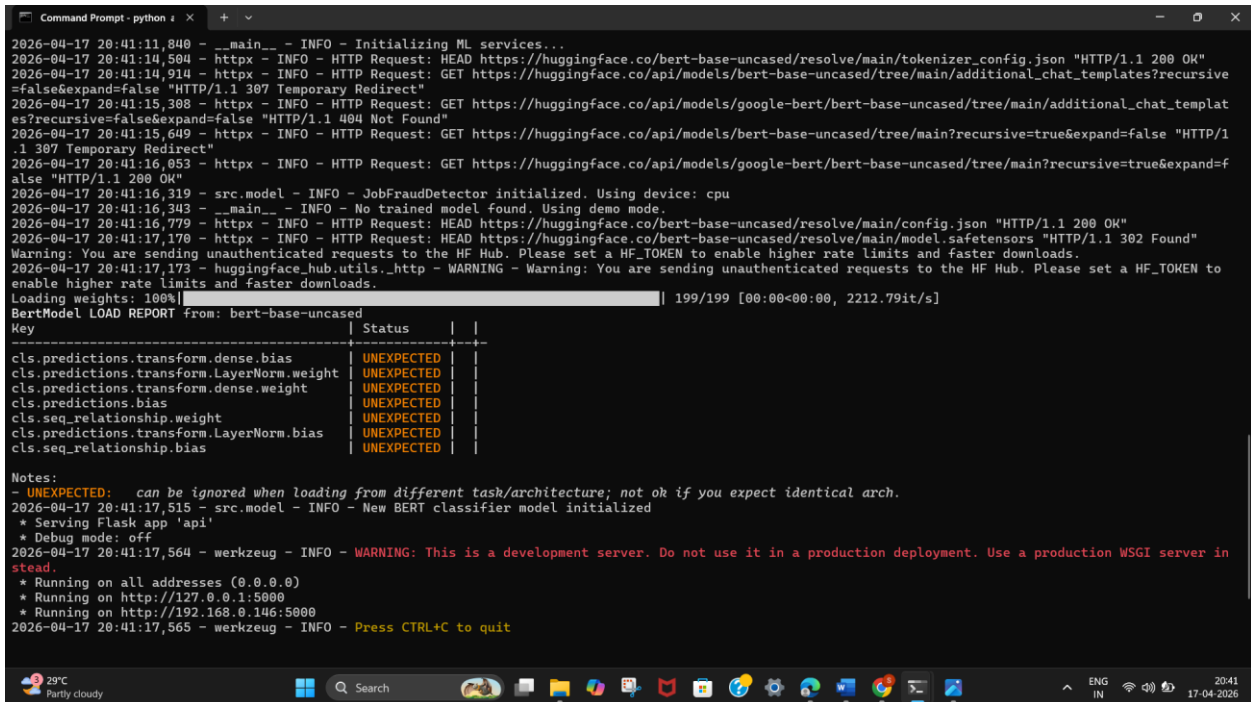
Local URL: http://localhost:8501
Network URL: http://192.168.0.146:8501
External URL: http://123.201.170.81:8501
```

Step-9: Run Backend Code

Move to backend folder:

Copy the path of folder in Backend file and paste it in the command prompt with cd

```
(venv) C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection>cd C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection\app\backend  
  
(venv) C:\Users\neelima\OneDrive\Desktop\online-recruitment-fraud-detection\app\backend>python api.py
```



```
2026-04-17 20:41:11,840 - __main__ - INFO - Initializing ML services...  
2026-04-17 20:41:14,504 - httpx - INFO - HTTP Request: HEAD https://huggingface.co/bert-base-uncased/resolve/main/tokenizer_config.json "HTTP/1.1 200 OK"  
2026-04-17 20:41:14,914 - httpx - INFO - HTTP Request: GET https://huggingface.co/api/models/bert-base-uncased/tree/main/additional_chat_templates?recursive=False&expand=False "HTTP/1.1 307 Temporary Redirect"  
2026-04-17 20:41:15,308 - httpx - INFO - HTTP Request: GET https://huggingface.co/api/models/google-bert/bert-base-uncased/tree/main/additional_chat_templates?recursive=False&expand=False "HTTP/1.1 404 Not Found"  
2026-04-17 20:41:15,649 - httpx - INFO - HTTP Request: GET https://huggingface.co/api/models/bert-base-uncased/tree/main?recursive=True&expand=False "HTTP/1.1 307 Temporary Redirect"  
2026-04-17 20:41:16,053 - httpx - INFO - HTTP Request: GET https://huggingface.co/api/models/google-bert/bert-base-uncased/tree/main?recursive=True&expand=False "HTTP/1.1 200 OK"  
2026-04-17 20:41:16,319 - src.model - INFO - JobFraudDetector initialized. Using device: cpu  
2026-04-17 20:41:16,343 - __main__ - INFO - No trained model found. Using demo mode.  
2026-04-17 20:41:16,779 - httpx - INFO - HTTP Request: HEAD https://huggingface.co/bert-base-uncased/resolve/main/config.json "HTTP/1.1 200 OK"  
2026-04-17 20:41:17,170 - httpx - INFO - HTTP Request: HEAD https://huggingface.co/bert-base-uncased/resolve/main/model.safetensors "HTTP/1.1 302 Found"  
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.  
2026-04-17 20:41:17,173 - huggingface_hub.utils._http - WARNING - Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.  
Loading weights: 100% | 199/199 [00:00<00:00, 2212.79it/s]  
BertModel LOAD REPORT from: bert-base-uncased  
Key | Status  
--- | ---  
cls.predictions.transform.dense.bias | UNEXPECTED  
cls.predictions.transform.LayerNorm.weight | UNEXPECTED  
cls.predictions.transform.dense.weight | UNEXPECTED  
cls.predictions.bias | UNEXPECTED  
cls.seq_relationship.weight | UNEXPECTED  
cls.predictions.transform.LayerNorm.bias | UNEXPECTED  
cls.seq_relationship.bias | UNEXPECTED  
  
Notes:  
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.  
2026-04-17 20:41:17,515 - src.model - INFO - New BERT classifier model initialized  
* Serving Flask app 'api'  
* Debug mode: off  
2026-04-17 20:41:17,564 - werkzeug - INFO - WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on all addresses (0.0.0.0)  
* Running on http://127.0.0.1:5000  
* Running on http://192.168.0.146:5000  
2026-04-17 20:41:17,565 - werkzeug - INFO - Press CTRL+C to quit
```

5.4 System Execution

The developed system, Job Guard AI, was successfully executed using a combination of Streamlit for the frontend and Flask for the backend. The system is designed to detect fraudulent job postings using a hybrid approach that combines a BERT-based deep learning model with rule-based analysis. The execution process demonstrates how the system accepts user input, processes it, and provides accurate predictions along with explanations.

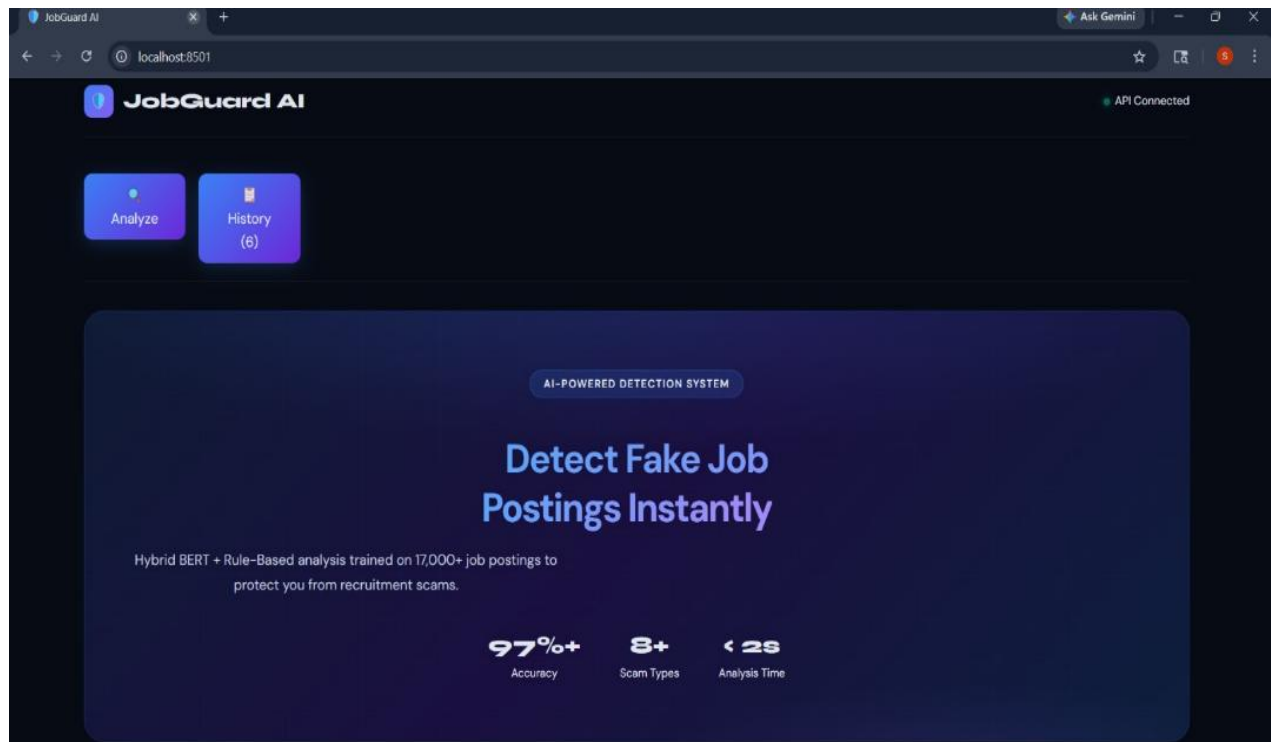
The system execution involves running the backend server and launching the frontend interface. Once the application is started, the user can interact with the system through a web browser.

The user can:

- Enter job descriptions manually
- Upload job posting images
- Perform quick or detailed analysis
- View past analysis history

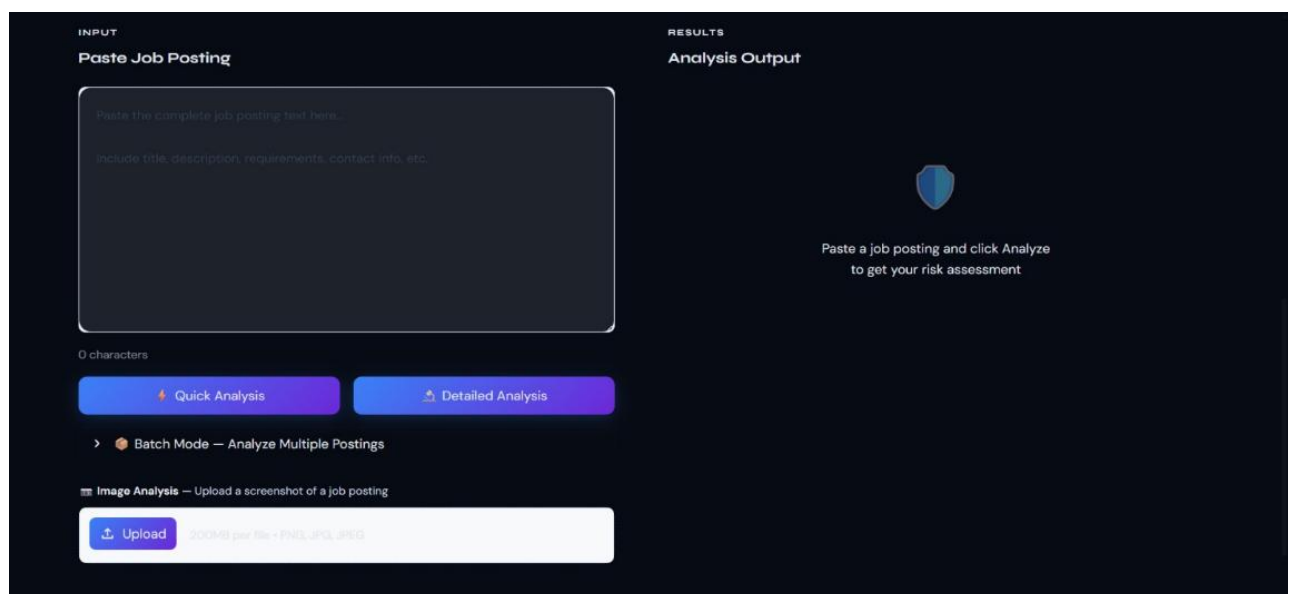
The system processes the input and displays results including fraud risk, safe score, and explanations.

1. Home Page of the System

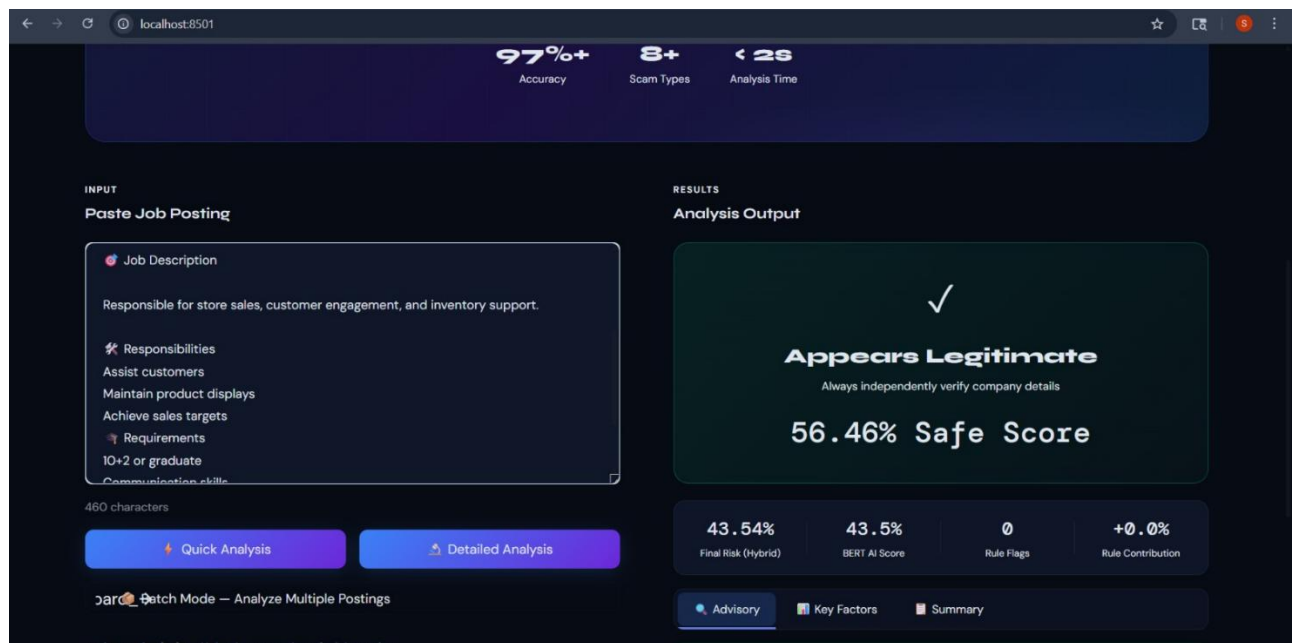


The above screen shows the main interface of the Job Guard AI system. It provides navigation options such as "Analyze" and "History". The system highlights its AI-powered detection capability, including accuracy, number of scam types detected, and fast processing time. This serves as the entry point for users.

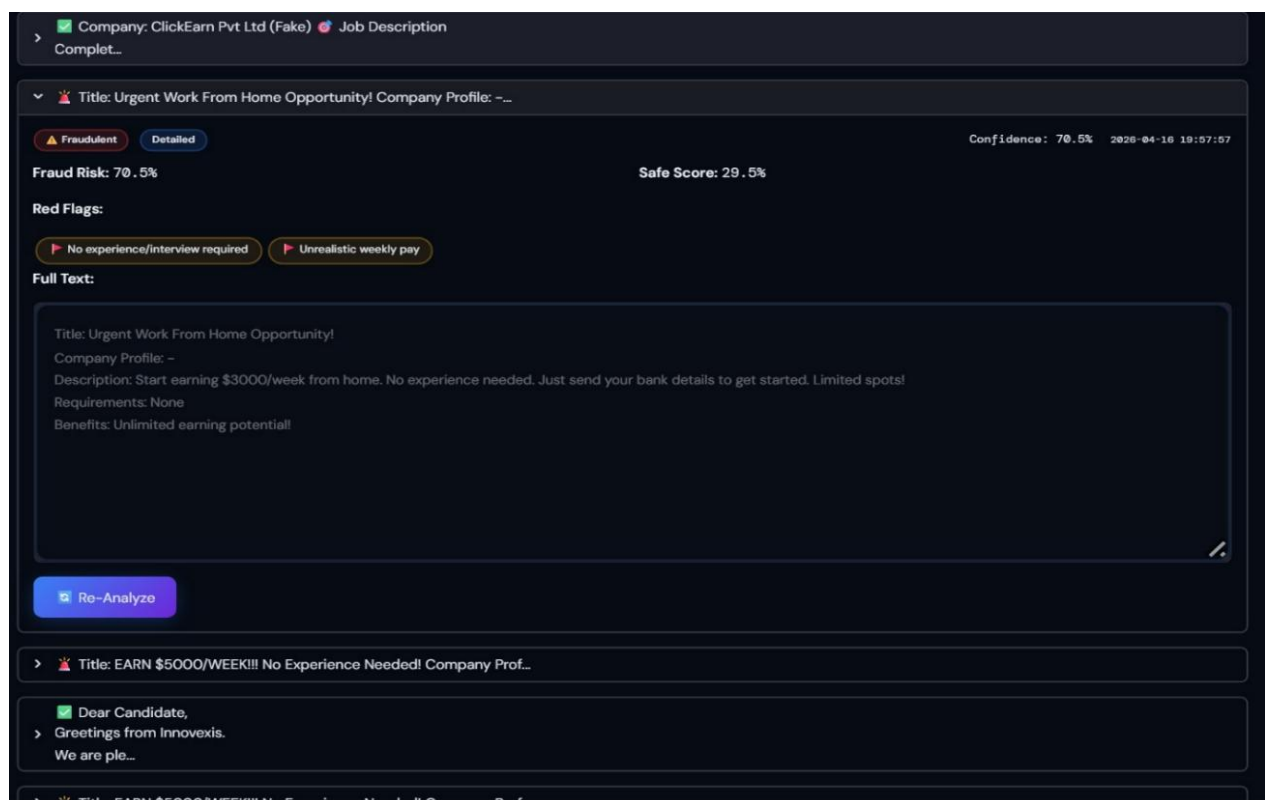
2. Job Input Interface



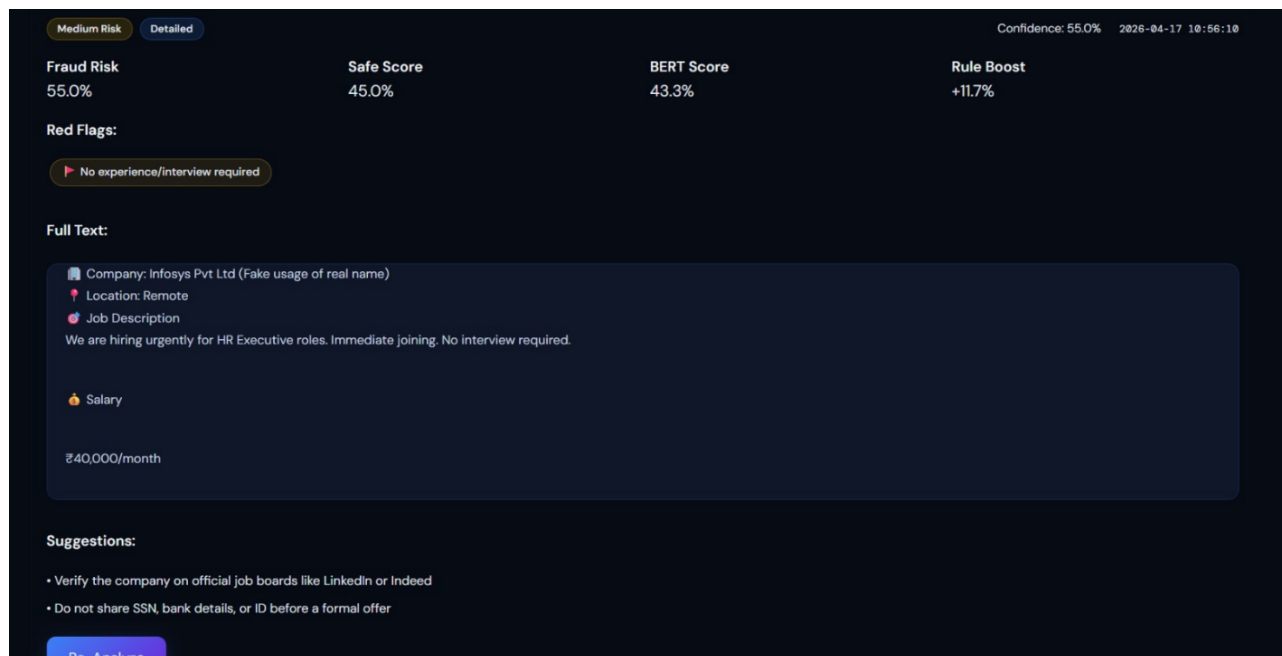
This interface allows users to paste job descriptions into the text box. Users can choose between "Quick Analysis" and "Detailed Analysis". Additional features include batch processing and image upload for analyzing job postings from screenshots. This ensures flexibility in input methods.



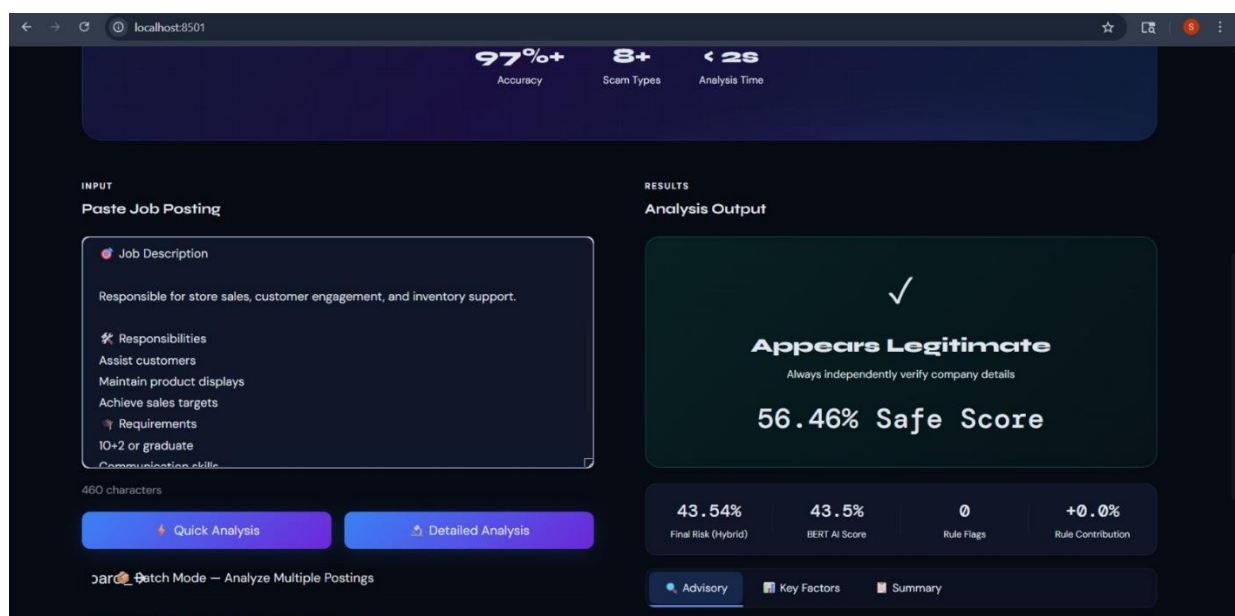
3. Fraudulent Job Detection Output



The above result shows a job posting identified as fraudulent. The system provides a fraud risk score and highlights red flags such as unrealistic salary and lack of interview requirements. This helps users quickly identify suspicious job postings.



4. Legitimate Job Detection Output



This screen shows a job classified as legitimate. The system provides a high confidence score and confirms that the job does not contain suspicious elements. This builds trust in the system's predictions.

5. Detailed Analysis Output

The system provides detailed metrics including:

- Fraud Risk Score

Company: ClickEarn Pvt Ltd (Fake)Job Description

Complet...

Title: Urgent Work From Home Opportunity! Company Profile: -...

FraudulentDetailed

Confidence: 70.5%2026-04-16 19:57:57

Fraud Risk: 70.5%Safe Score: 29.5%

Red Flags:

No experience/interview requiredUnrealistic weekly pay

Full Text:

Title: Urgent Work From Home Opportunity!
Company Profile: -
Description: Start earning \$3000/week from home. No experience needed. Just send your bank details to get started. Limited spots!
Requirements: None
Benefits: Unlimited earning potential!

Re-Analyze

Title: EARN \$5000/WEEK!!! No Experience Needed! Company Prof...

Dear Candidate,
Greetings from Innovaxis.
We are ple...

Fraud Risk55.0%

Safe Score45.0%

BERT Score43.3%

Rule Boost+11.7%

Red Flags:

No experience/interview required

Full Text:

Company: Infosys Pvt Ltd (Fake usage of real name)

Location: Remote

Job Description

We are hiring urgently for HR Executive roles. Immediate joining. No interview required.

Salary

₹40,000/month

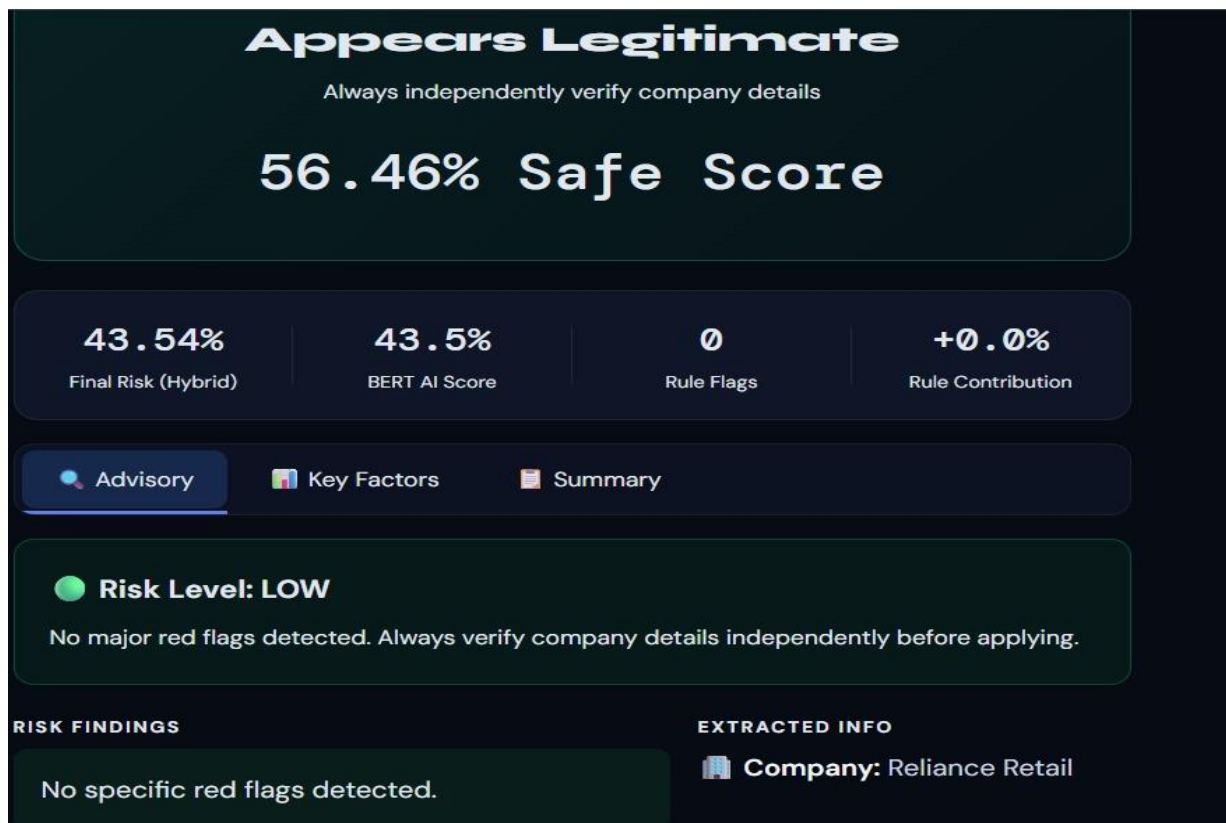
Suggestions:

Verify the company on official job boards like LinkedIn or Indeed

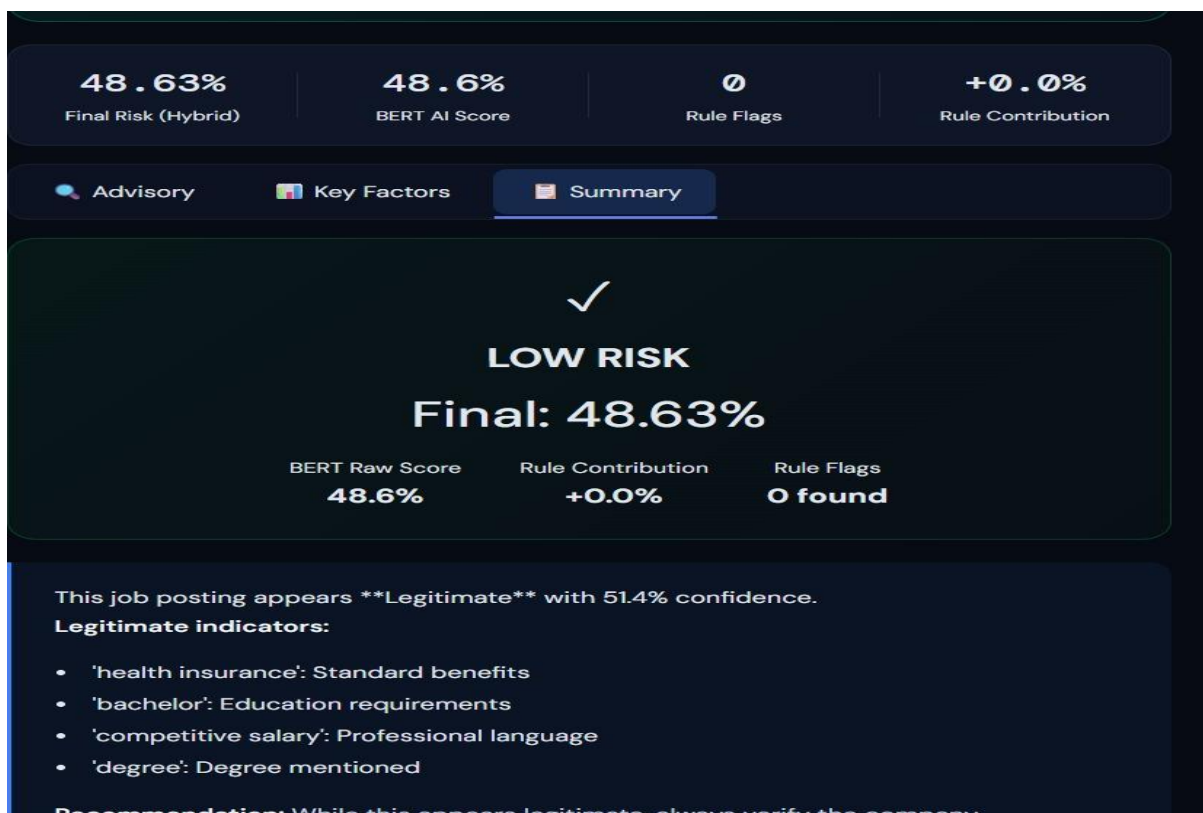
Do not share SSN, bank details, or ID before a formal offer

Re-Analyze

- Safe Score

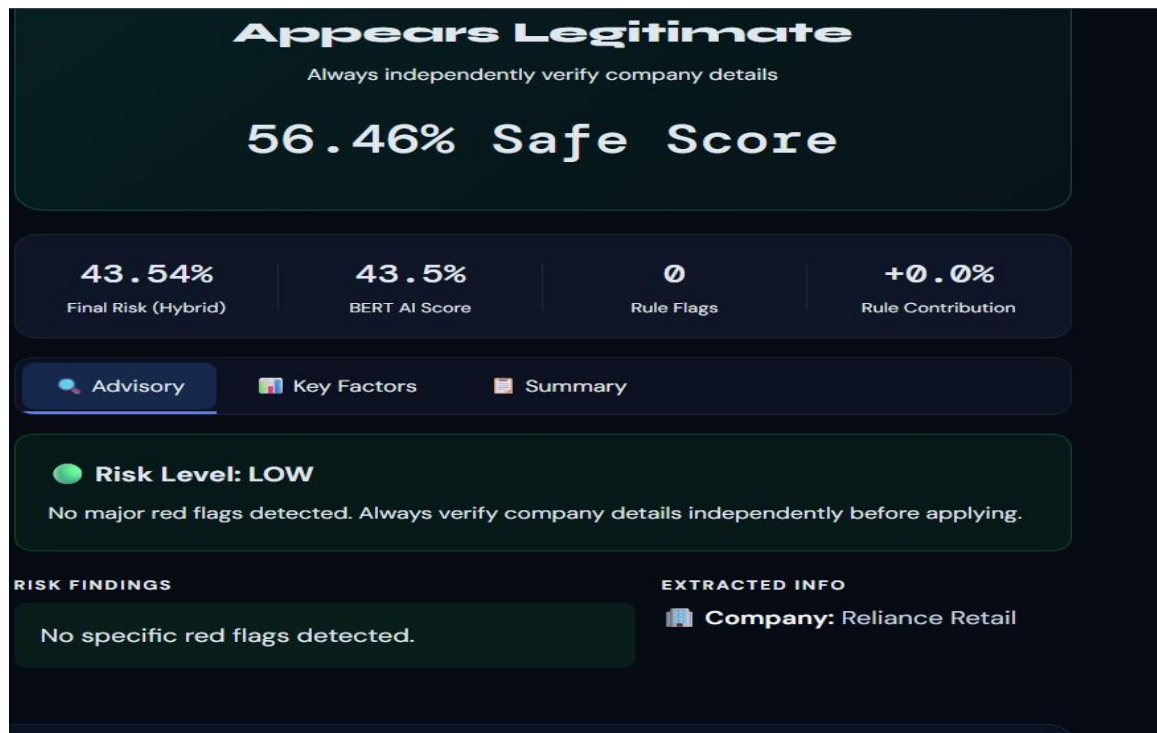


- BERT Model Score
- Rule-Based Contribution



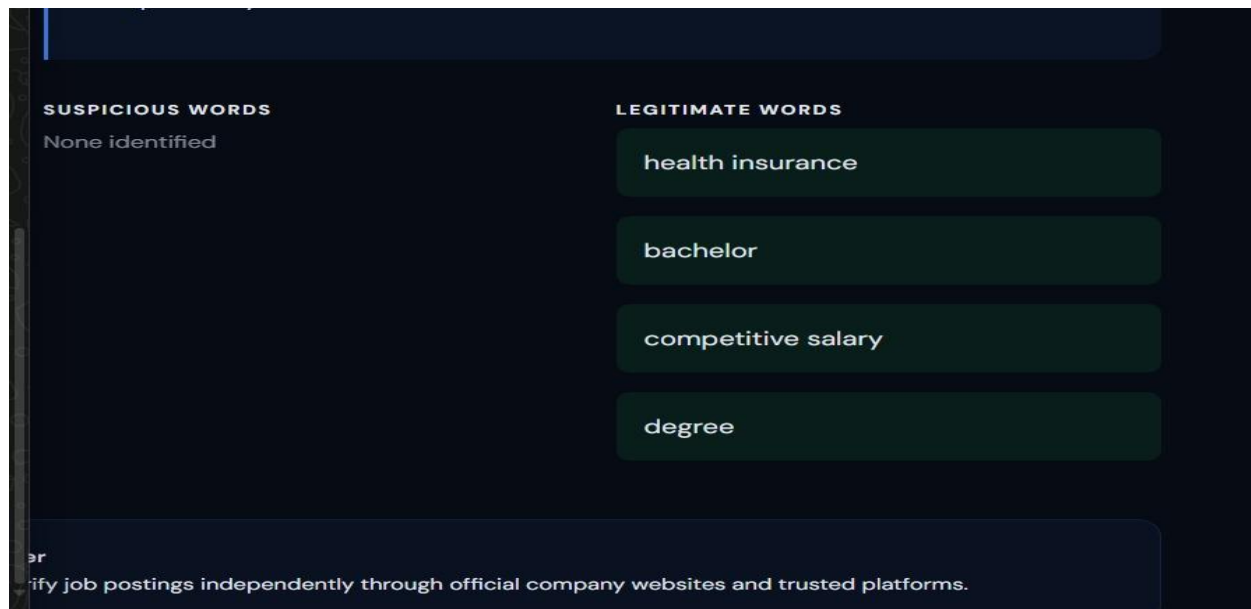
These metrics help users understand how the final prediction is generated.

6. Advisory and Recommendations



The system provides useful suggestions such as verifying company details, checking job authenticity, and avoiding sharing sensitive information. This enhances user awareness and safety.

7. Suspicious and Legitimate Words Analysis



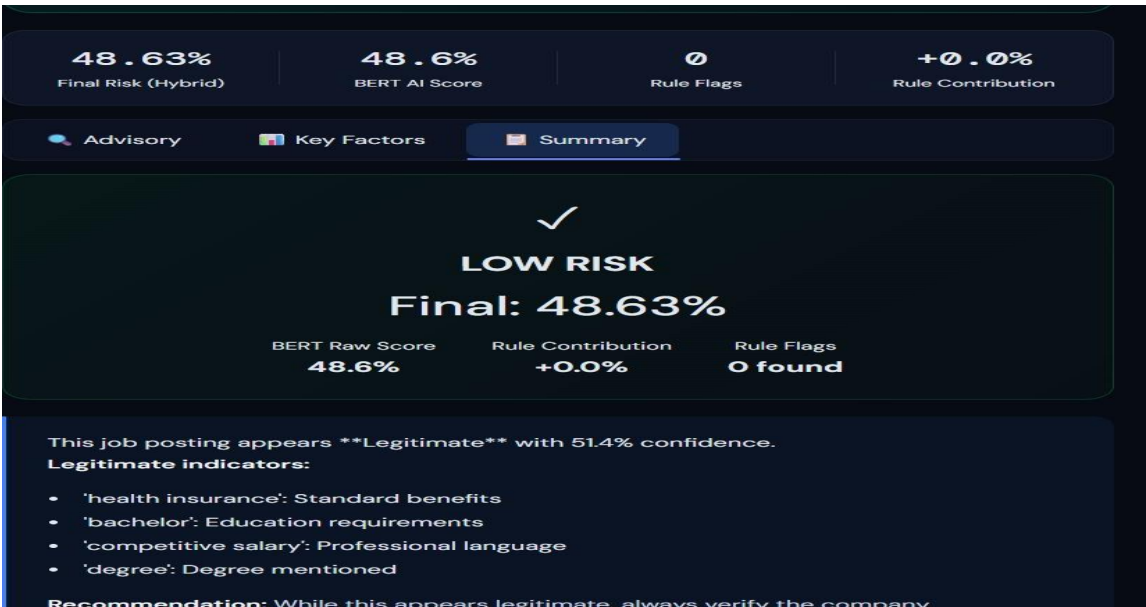
The system identifies suspicious and legitimate words present in the job posting. This helps in understanding which words influenced the prediction and improves transparency.

8. Word Impact Visualization



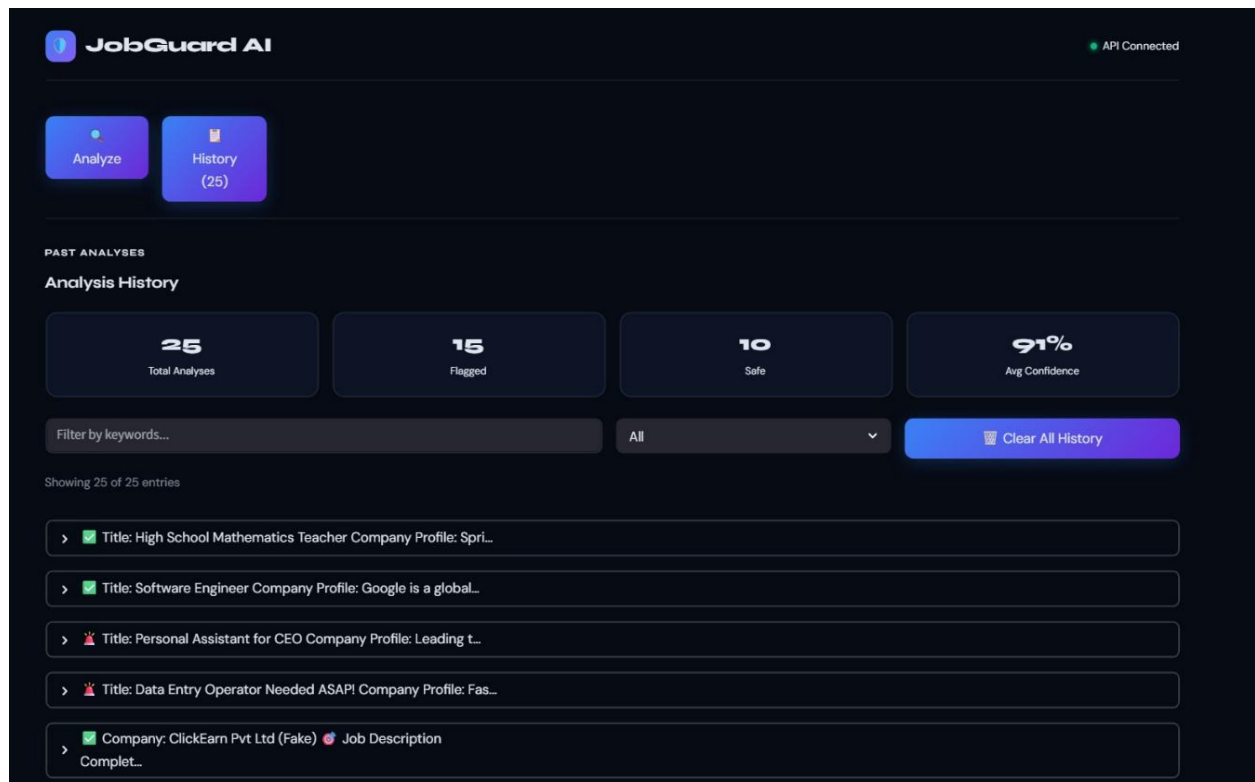
This visualization shows how different words contribute to the prediction. It highlights the importance of specific keywords in determining whether a job is fraudulent or legitimate

9. Summary Output



The summary section provides a clear conclusion of the analysis, indicating whether the job posting is legitimate or fraudulent along with a confidence score. It simplifies the decision-making process for users.

10. Analysis History Dashboard



The system maintains a history of previously analyzed job postings. It displays statistics such as total analyses, flagged jobs, safe jobs, and average confidence. This helps users track and review past predictions effectively

5.4 Results

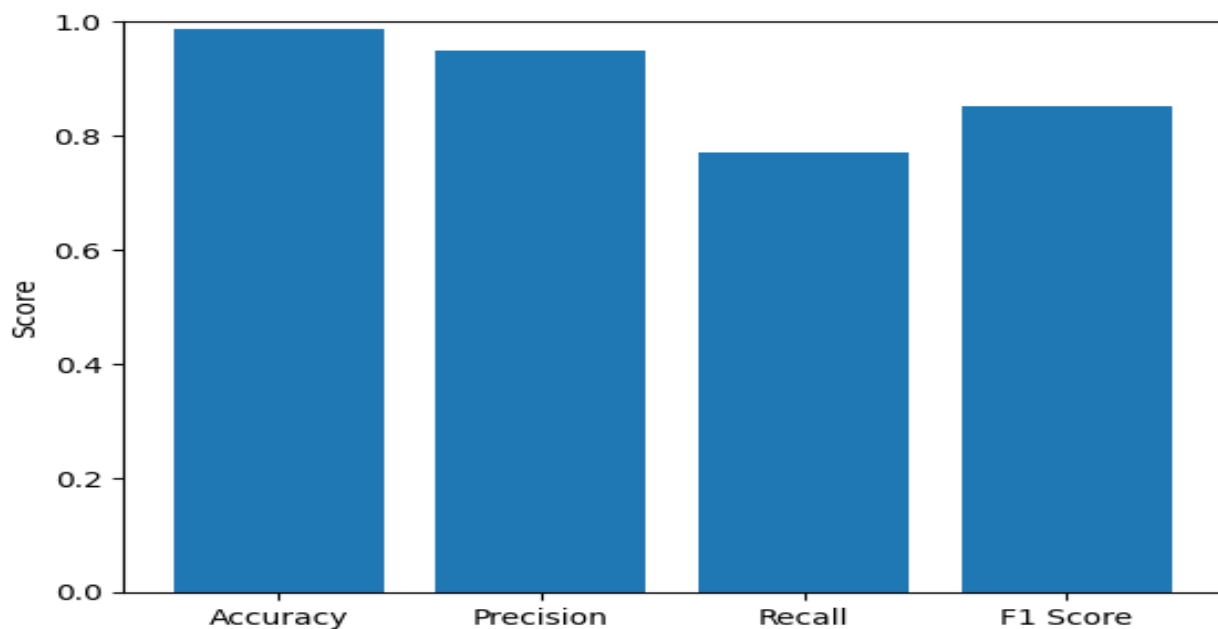


Fig vi : Model Performance Metrics

6. CONCLUSION

The Fake Job Detection System successfully addresses the growing issue of fraudulent job postings in online recruitment platforms by providing an intelligent, accurate, and user-friendly solution. The system integrates advanced technologies such as a BERT-based deep learning model, rule-based detection, OCR for image processing, and LIME for explainable AI, resulting in a robust hybrid approach capable of detecting both obvious and sophisticated fraud patterns.

The implementation demonstrates that combining contextual understanding through deep learning with explicit rule-based checks significantly improves detection accuracy. The system effectively identifies common fraud indicators such as unrealistic salary offers, requests for upfront payments, and suspicious communication methods, while also detecting subtle scams that mimic legitimate job postings. The inclusion of OCR further enhances the system's capability by enabling analysis of image-based job advertisements, which are commonly used by fraudsters to bypass traditional detection systems.

Another important achievement of the system is its explainability feature. By using LIME, the system provides clear and interpretable explanations for its predictions, increasing transparency and user trust. The user-friendly interface and support for batch processing make the system practical and accessible for real-world usage.

Comprehensive testing at multiple levels ensures that the system is reliable, efficient, and capable of handling different types of inputs. The results indicate that the system performs well in detecting fraudulent job postings and can be effectively deployed to enhance safety in online recruitment environments.

In conclusion, this project presents a scalable and effective solution for fake job detection, contributing to the prevention of online recruitment fraud and helping users make safer and more informed decisions.

7. REFERENCES

- [1] P. Kaur, “E-recruitment: A conceptual study,” *Int. J. Appl. Res.*, vol. 1, no. 8, pp. 78–82, 2015.
- [2] C. S. Anita, P. Nagarajan, G. A. Sairam, P. Ganesh, and G. Deepakkumar, “Fake job detection and analysis using machine learning and deep learning algorithms,” *Revista Gestão Inovação e Tecnologias*, vol. 11, no. 2, pp. 642–650, Jun. 2021.
- [3] A. Raza, S. Ubaid, F. Younas, and F. Akhtar, “Fake e job posting prediction based on advance machine learning approachs,” *Int. J. Res. Publication Rev.*, vol. 3, no. 2, pp. 689–695, Feb. 2022.
- [4] Online Fraud. Accessed: Jun. 19, 2022. [Online]. Available: <https://www.cyber.gov.au/acsc/report>
- [5] J. Howington, “Survey: More millennials than seniors victims of job scams,” Flexjobs, CO, USA, Sep. 2015. Accessed: Jan. 2024 [Online]. Available: www.flexjobs.com/blog/post/survey-results-millennials-seniors-victims-job-scams
- [6] Report Cyber. Accessed: Jun. 25, 2022. [Online]. Available: <https://www.actionfraud.police.uk/>
- [7] S. Vidros, C. Kolias, G. Kambourakis, and L. Akoglu, “Automatic detection of online recruitment frauds: Characteristics, methods, and a public dataset,” *Future Internet*, vol. 9, no. 1, p. 6, Mar. 2017.
- [8] S. Dutta and S. K. Bandyopadhyay, “Fake job recruitment detection using machine learning approach,” *Int. J. Eng. Trends Technol.*, vol. 68, no. 4, pp. 48–53, Apr. 2020.
- [9] B. Alghamdi and F. Alharby, “An intelligent model for online recruitment fraud detection,” *J. Inf. Secur.*, vol. 10, no. 3, pp. 155–176, 2019.
- [10] S. Lal, R. Jiaswal, N. Sardana, A. Verma, A. Kaur, and R. Mourya, “ORFDetector: Ensemble learning based online recruitment fraud detection,” in *Proc. 12th Int. Conf. Contemp. Comput. (IC3)*, Noida, India, Aug. 2019, pp. 1–5.
- [11] I. M. Nasser, A. H. Alzaanin, and A. Y. Maghari, “Online recruitment fraud detection using ANN,” in *Proc. Palestinian Int. Conf. Inf. Commun. Technol. (PICICT)*, Sep. 2021, pp. 13–17.
- [12] C. Lokku, “Classification of genuinity in job posting using machine learning,” *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 9, no. 12, pp. 1569–1575, Dec. 2021.
- [13] O. Nindyati and I. G. Bagus Baskara Nugraha, “Detecting scam in online job vacancy using behavioral features extraction,” in *Proc. Int. Conf. ICT Smart Soc. (ICISS)*, vol. 7, Bandung, Indonesia, Nov. 2019, pp. 1–4.
- [14] S. Kotsiantis, D. Kanellopoulos, and P. Pintelas, “Handling imbalanced datasets: A review,” *GESTS Int. Trans. Comput. Sci. Eng.*, vol. 30, no. 1, pp. 25–36, 2006.

- [15] M. Tavallaei, N. Stakhanova, and A. A. Ghorbani, "Toward credible evaluation of anomaly-based intrusion-detection methods," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 40, no. 5, pp. 516–524, Sep. 2010.
- [16] Y.-H. Liu and Y.-T. Chen, "Total margin based adaptive fuzzy support vector machines for multiview face recognition," in *Proc. IEEE Int. Conf. Syst., Man Cybern.*, Waikoloa, HI, USA, Oct. 2005, pp. 1704–1711.
- [17] M. A. Mazurowski, P. A. Habas, J. M. Zurada, J. Y. Lo, J. A. Baker, and G. D. Tourassi, "Training neural network classifiers for medical decision making: The effects of imbalanced datasets on classification performance," *Neural Netw.*, vol. 21, nos. 2–3, pp. 427–436, Mar. 2008.
- [18] Y. Li, G. Sun, and Y. Zhu, "Data imbalance problem in text classification," in *Proc. 3rd Int. Symp. Inf. Process.*, Luxor, Egypt, Oct. 2010, pp. 301–305.
- [19] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.
- [20] S. U. Habiba, Md. K. Islam, and F. Tasnim, "A comparative study on fake job post prediction using different data mining techniques," in *Proc. 2nd Int. Conf. Robot., Electr. Signal Process. Techn. (ICREST)*, Dhaka, Bangladesh, Jan. 2021, pp. 543–546.